

**Escuela Técnica Superior de Ingeniería Informática y
Telecomunicaciones UGR**



DISEÑO Y DESARROLLO DE SISTEMAS DE INFORMACIÓN

Curso 2025/2026

Práctica 3:
Implementación del Sistema de Información

Autores:

Aitor de la Iglesia García
Juan Manuel Fernández García
Daniel Hidalgo Chica
Roberto González Lugo
Elsa Rodríguez Macmichael

Índice

DISEÑO Y DESARROLLO DE SISTEMAS DE INFORMACIÓN.....	1
Práctica 3:.....	1
Implementación del Sistema de Información.....	1
Índice.....	2
1. Descripción del SI.....	3
2. Requisitos Funcionales.....	7
2.1. Gestión de perfiles (Elsa Rodríguez).....	7
2.2. Gestión de productos (Roberto González).....	12
2.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo).....	16
2.4. Gestión de ventas (Juan Manuel Fernández).....	20
2.5. Gestión de mensajes (Aitor de la Iglesia).....	24
3. Restricciones semánticas.....	28
3.1. Gestión de perfiles (Elsa Rodríguez).....	28
3.2. Gestión de productos (Roberto González).....	30
3.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo).....	33
3.4. Gestión de ventas (Juan Manuel Fernández).....	33
3.5. Gestión de mensajes (Aitor de la Iglesia).....	34
4. Diagramas y paso a tablas.....	36
4.1. Caja Negra.....	36
4.2. Armazón.....	36
4.3. DFD1s.....	37
4.4. Esquemas externos.....	41
4.5. Paso a tablas.....	55
4.6. Fusión de tablas.....	56
4.7. Dependencias funcionales.....	57
4.8. Normalización.....	58
5. Creación de tablas.....	61
6. Descripción de las transacciones.....	64
6.1. Gestión de Perfiles (Elsa Rodríguez).....	64
6.2. Gestión de Productos (Roberto González).....	65
6.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo).....	66
6.4. Gestión de Ventas (Juan Manuel Fernández).....	67
6.5. Gestión de Mensajes (Aitor de la Iglesia).....	68
7. Disparadores.....	70
8. Software elegido.....	88
9. Datos sensibles y aspectos legales.....	88

1. Descripción del SI

TikiTún es un sistema de información multiusuario orientado a la compraventa de productos entre particulares. Su funcionamiento se basa en la publicación de artículos por parte de los usuarios, la consulta de dichos artículos a través de un sistema de búsqueda y recomendaciones, y la interacción directa entre compradores y vendedores para llegar a acuerdos de transacción.

El sistema está estructurado en varios subsistemas que se integran:

- **Gestión de perfiles** (Elsa Rodríguez): Incluye el registro, modificación y baja de usuarios, así como la gestión de preferencias y de un monedero virtual. También la consulta de usuarios (tanto el propio como otros).
- **Gestión de productos** (Roberto González): Permite dar de alta, modificar, eliminar, consultar y promocionar artículos.
- **Feed de recomendaciones, búsquedas y favoritos** (Daniel Hidalgo): Muestra al usuario productos que podrían serle de interés, permite la búsqueda con filtrado por cadena de caracteres y categoría, y controla productos guardados como favoritos.
- **Gestión de ventas** (Juan Manuel Fernández): Regula los procesos de compra directa y negociación mediante contraofertas.
- **Mensajería** (Aitor de la Iglesia): Facilita la comunicación entre usuarios vinculada a los productos.

De cada usuario se almacena un **nombre completo** (en una cadena de 60 caracteres), **nombre de usuario** (en una cadena de 15 caracteres), **contraseña** (serie de entre 8 y 15 caracteres con obligatoriedad de contener una mayúscula, una minúscula, un número y un carácter especial, y sin espacios), **correo electrónico** (cadena de caracteres con el formato usual), **ubicación** (tupla latitud, longitud), **rango** en kilómetros (número real positivo con dos cifras decimales), **saldo** del monedero en euros (número real positivo con dos cifras decimales), **productos ofertados** (lista de identificadores de productos), **productos favoritos** (lista de identificadores de productos), **preferencias de categorías** (lista de entre 1 y 6 cadenas de caracteres escogidas de entre un conjunto de categorías preestablecidas por el sistema), **valoración media** (número real entre 1 y 5 con un decimal, con saltos de 0.5 en 0.5), y un campo **cuenta_eliminada** (booleano, *true* si el usuario se ha dado de baja).

Para dar de alta a cada usuario se le pedirá que introduzca nombre, nombre de usuario (único), contraseña, correo electrónico (único); que seleccione su ubicación y rango de interés (en km); que marque entre 1 y 6 categorías de preferencia; que confirme su mayoría de edad; y que acepte la política de privacidad y condiciones de uso de la plataforma.

Un usuario podrá acceder a la información de su perfil y editarla (datos personales y preferencias), añadir saldo a su monedero o transferirlo a una cuenta bancaria, visitar otros perfiles, y por último, darse de baja de la plataforma. En este último caso, se actualizan a valores nulos todos sus campos personales excepto el nombre de usuario (clave primaria) y

este deja de poder acceder a la plataforma, además de anularse todos sus productos ofertados. Darse de baja sólo será posible si no tiene ventas activas, esto es, no es vendedor ni comprador de un producto comprado del que todavía no se ha confirmado su recepción, ni tampoco ha realizado ninguna contraoferta que siga activa.

Un producto es un artículo en venta publicado por un usuario. De cada producto se almacenará un **Id_producto** (unsigned int), **título** (cadena de hasta 80 caracteres), **descripción** (cadena de hasta 500 caracteres), **precio** en euros (número real positivo con dos cifras decimales), **categoría** (elemento perteneciente a una estructura jerárquica de categorías), **grado de promoción** (número real entre 0 y 1 con dos decimales), e **imagen** (1 fichero en formato .jpg de 1024x1024 píxeles) Además de un valor booleano que indica si el producto está disponible para su venta (no ha sido solicitada eliminación ni se ha vendido). También se guardará cuánta gente lo tiene en favoritos (int).

Para **dar de alta un producto**, el usuario deberá introducir su título, descripción, precio, categoría (que se elegirá de una lista cerrada de opciones) y adjuntar una imagen. El sistema validará que el precio sea mayor que cero. El identificador del producto (**Id_producto**) se asignará automáticamente y no será visible ni editable por el usuario.

El usuario podrá **modificar** los datos de un producto que él mismo haya publicado (título, descripción, precio, categoría e imagen adjunta) o **eliminarlo** (marcarlo como no disponible y hacerlo invisible a efectos prácticos) cuando lo desee. Los productos publicados podrán ser consultados por otros usuarios a través del feed, búsquedas y perfiles de vendedores.

Opcionalmente, el usuario podrá **promocionar** un producto. El sistema incrementará la visibilidad del producto en el feed en función del valor registrado en el campo **grado de promoción**. Para establecer este valor, el usuario deberá abonar una cantidad de dinero de su monedero, proporcional al nivel de promoción seleccionado.

El **grado de promoción** se reducirá automáticamente en 0,1 unidades cada 24 horas. Esto implica que la visibilidad del producto también disminuirá progresivamente con el paso de los días. Un producto que se promocioe con el valor máximo (1,0 en la escala de 0 a 1) alcanzará el nivel mínimo (0) en diez días, perdiendo visibilidad de manera gradual durante ese periodo. Respecto al coste de la promoción, se calculará de la siguiente forma:
$$\text{Coste_promoción} = \text{grado_promoción_deseado} * 0,1 * \text{precio_producto}$$

Para desencadenar la compra de un producto el mismo se le presentará al usuario o bien en una pestaña de recomendaciones (*feed*) o tras hacer una búsqueda en otra pestaña dedicada.

Para la gestión de la pestaña de recomendaciones, se tendrán en cuenta algunos datos ya presentados como las categoría de los productos, las categorías asociadas como preferidas al perfil del usuario, además de su ubicación (como latitud y longitud) y su rango de disponibilidad (en km), la puntuación del perfil de los vendedores y el número de usuarios que tengan un producto en favoritos.

Cuando el usuario abra la pestaña de recomendaciones, se mostrarán sólo productos cuyo vendedor esté lo bastante cerca del usuario (en tanto que la distancia entre

vendedor y usuario sea menor que la suma de los rangos de disponibilidad de éstos) y que además estén bajo alguna de las categorías seleccionadas como “preferidas” por el usuario ordenados según el siguiente criterio: se muestran primero los productos que pertenezcan a alguna de las categorías “preferidas” por el usuario, ordenando por grado de promoción de los productos, si hay empate, se ordena por número de personas que tengan el producto en favoritos (descendentemente), y en caso de doble empate, aleatoriamente.

Por otro lado también se controlarán los productos que el usuario haya marcado como “favoritos”. De cada usuario se guarda una lista de productos que son considerados sus “favoritos”. En particular, tras consultar un producto, un usuario puede decidir marcarlo como favorito o quitarlo de entre sus favoritos, quedando esto reflejado en la base de datos. Además, un usuario podrá consultar qué productos figuran entre sus favoritos. Naturalmente, solo aparecerán en la consulta de los favoritos, y solo podrán marcarse como favoritos los productos que en ese momento estén “disponibles” (que no hayan sido vendidos ni eliminados).

En cuanto a la gestión de las búsquedas, tan solo se requieren los datos ya presentados en el documento del nombre, el precio, la promoción, y la categoría de cada producto; y la puntuación de cada perfil, además de la ubicación y rango de disponibilidad de cada usuario (al potencial comprador se le mostrarán solo productos que estén “lo bastante cerca” según el mismo criterio detallado en la pestaña de recomendaciones).

Para buscar, el usuario puede seleccionar (opcionalmente) la categoría de búsqueda, y una cadena de caracteres para filtrar los productos por nombre de producto. Así, en cuanto desencadene la acción de buscar (con un botón) se realiza en la base de datos una consulta de la que resultan los productos cuyos nombres contengan la cadena de caracteres buscada (*fuzzy search*) y que sean de la categoría seleccionada.

Hay dos opciones en cuanto al orden en el que se presentan los productos. El establecido por defecto, es el siguiente: los productos que coincidan con la búsqueda, ordenados por puntuación de perfil. En caso de que se seleccione explícitamente, también se podrán mostrar los productos por precio, ascendente o descendentemente, y desempataando por puntuación de perfil en orden descendente.

De una venta almacenamos la **clave del producto**, los **identificadores de los usuarios implicados**, la **fecha de la venta**, el **precio final de la venta** y el **estado de recepción del producto** (booleano que indica si se ha recibido el producto).

Además, también se almacenan las **contraofertas asociadas a cada producto**, guardando la **clave del vendedor**, la **clave del producto** y el **precio ofrecido en la contraoferta** realizada.

Bajo la descripción de la información de cada producto se encuentran dos botones:

El primero de ellos permite **realizar la compra del producto directamente** por el precio establecido por el vendedor. Al pulsar dicho botón se pide la confirmación del usuario para realizar dicha compra, y en caso disponer de saldo suficiente se **retira del monedero del comprador. El producto pasa a un estado pendiente de confirmación** hasta que el comprador recibe el producto, siendo ese el momento en el que se transfiere el dinero al monedero del vendedor.

Si el comprador no tuviera saldo suficiente la compra se cancela y se notifica al usuario que no tiene saldo suficiente para realizar la compra.

El segundo de ellos permite al comprador **realizar una contraoferta por el producto** por un precio estrictamente inferior al estipulado por el vendedor. Además, no se podrán hacer contraofertas por un precio que supere el saldo disponible en el monedero del comprador.

Una vez establecida la contraoferta se pide la confirmación del usuario y **se comprueba si la contraoferta es viable**, y en caso de que lo sea, se notifica el éxito de la operación al comprador y de la nueva contraoferta al vendedor.

En caso de no cumplirse el requisito anterior, se cancela la operación y se notifica al comprador que su contraoferta no puede realizarse.

Una vez recibida la contraoferta, el vendedor puede decidir si aceptarla o rechazarla. Si la acepta, la compra se efectúa en los términos descritos antes, y si la rechaza se notifica al comprador que la contraoferta ha sido rechazada y se elimina dicha contraoferta de las asociadas al producto.

En cualquier caso, una vez que se ha efectuado la compra se notifica tanto al comprador como al vendedor que se ha realizado dicha compra. Además, al usuario se le pedirá que **evalúe el artículo comprado** mediante un sistema de puntajes de 0 a 5 estrellas con saltos de media estrella.

En caso del vendedor, éste dispone de la capacidad de **consultar todas las contraofertas recibidas para cada uno de los productos** que tenga en venta.

Para facilitar la comunicación entre comprador y vendedor en relación a la venta de un producto se almacena una **conversación** (lista de mensajes, nombres de usuario del vendedor y comprador, id del producto), compuesta por **mensajes** (cadenas de 200 caracteres, fichero (opcional), emisor), que tienen asociados una **fecha de envío** (tipo Date) y una **marca para indicar si el mensaje se ha leído** (booleano). Los mensajes entrantes se **notifican** al correo del usuario.

El usuario podrá **enviar un mensaje a otro usuario** siempre en relación a la compra de un producto. Cuando la compra finalice y sea confirmada la **conversación** pasará a ser **archivada**. El usuario también podrá **consultar** todas las **conversaciones** que tenga abiertas. También se podrán **buscar mensajes** concretos entre todas las conversaciones que tenga el usuario. A cada mensaje se le puede **añadir un archivo adjunto**. Cuando un usuario recibe un mensaje, se le **notifica** a su correo electrónico.

2. Requisitos Funcionales

2.1. Gestión de perfiles (Elsa Rodríguez)

RF1.1: Dar de alta al usuario.

Entrada:

- Agente externo: usuario.
- Acción: solicitar inserción.
- Requisito de datos de entrada: RDE1.1.

BD: Requisito de datos de escritura: RDW1.1.

Salida:

- Agente externo: usuario.
- Acción: confirmación de registro o mensaje de error.
- Requisito de datos de salida: ninguno.

RDE1.1: Datos de entrada de alta de usuario

- Nombre completo: cadena de caracteres (60)
- Nombre de usuario: cadena de caracteres (15), único
- Contraseña: cadena de entre 8 y 15 caracteres (con al menos una mayúscula, una minúscula, un número y un carácter especial, sin espacios)
- Correo electrónico: cadena con formato válido, único
- Ubicación: tupla (latitud, longitud)
- Rango en kilómetros: número real positivo con dos decimales
- Categorías de preferencia: lista de entre 1 y 6 elementos de una jerarquía de categorías.
- Confirmación de mayoría de edad: booleano
- Aceptación de política de privacidad y condiciones de uso: booleano

RDW1.1: Datos almacenados de usuario

Los mismos datos que RDE1.1 excepto la confirmación de mayoría de edad y la aceptación de política de privacidad, más los siguientes campos:

- Saldo monedero: número real positivo con dos decimales (por defecto 0,00)
- Valoración media: real entre 1 y 5 con un decimal (por defecto sin valor o 0)
- cuenta_eliminada: booleano (por defecto *false*)

RF1.2: Consultar perfil de usuario.

Entrada:

- Agente externo: usuario.

- Acción: solicitar consulta de perfil (propio o ajeno).
- Requisito de datos de entrada: RDE1.2.

BD: Requisito de datos de lectura: RDR1.2.

Salida:

- Agente externo: usuario.
- Acción: mostrar información del perfil o error (si usuario no existe o cuenta eliminada).
- Requisito de datos de salida: RDS1.2.

RDE1.2: Datos de entrada de consulta de perfil

- Nombre de usuario: cadena de caracteres (15).

RDR1.2.1: Datos leídos del perfil de usuario

- Nombre de usuario: cadena de caracteres (15)
- Saldo monedero: número real positivo con dos decimales (por defecto 0,00)
- Valoración media: real entre 1 y 5 con un decimal (por defecto sin valor o 0)
- cuenta_eliminada: booleano (por defecto *false*)

RDR1.2.2: Datos leídos de sus productos ofertados

- Nombre de usuario: cadena de caracteres (15).
- Lista de resúmenes de productos disponibles del usuario (id_producto, título, imagen, precio, disponible)
- Lista de id_productos de sus productos no disponibles (para tomar los que se hayan vendido)

RDR1.2.3: Datos leídos de sus productos vendidos

- Lista de ids_productos (no disponibles)
- Número total de valoraciones recibidas: número natural o 0

RDS1.2: Datos mostrados en pantalla

- Nombre completo, nombre de usuario, valoración media y resúmenes de productos ofertados.

RF1.3: Modificar perfil de usuario.

Entrada:

- Agente externo: usuario.
- Acción: solicitar modificación.
- Requisito de datos de entrada: RDE1.3.

BD: Requisito de datos de escritura: RDW1.3.

Salida:

- Agente externo: usuario.
- Acción: confirmación de modificación o error.

RDE1.3: Datos de entrada de modificación de perfil

- Nombre de usuario: cadena de caracteres (15)
- Nombre completo (opcional)
- Correo electrónico (opcional, formato válido y único)
- Ubicación (opcional: latitud, longitud)
- Rango (opcional)
- Categorías de preferencia (entre 1 y 6)

RDR1.3: Datos leídos del perfil de usuario

- Nombre de usuario: cadena de caracteres (15)
- cuenta_eliminada: booleano

RDW1.3: Datos actualizados en la BD

- Los mismos datos que RDE1.3, actualizados en el perfil correspondiente.

RF1.4: Añadir saldo al monedero.

Entrada:

- Agente externo: usuario.
- Acción: solicitar incremento de saldo.
- Requisito de datos de entrada: RDE1.4.

BD: Requisito de datos de escritura: RDW1.4.

Salida:

- Agente externo: usuario.
- Acción: confirmación del nuevo saldo o error de transacción.

RDE1.4: Datos de entrada de recarga

- Nombre de usuario: cadena de caracteres (15)
- Cantidad: número real positivo con dos decimales.

RDR1.4: Datos leídos del perfil de usuario

- Nombre de usuario: cadena (15)

- cuenta_eliminada: booleano
- Saldo monedero: número real positivo con dos decimales

RDW1.4: Datos almacenados

- Nombre de usuario: cadena (15)
- Nuevo saldo = saldo anterior + cantidad añadida.

RF1.5: Transferir saldo a cuenta bancaria.

Entrada:

- Agente externo: usuario.
- Acción: solicitar retirada de saldo.
- Requisito de datos de entrada: RDE1.5.

BD: Requisito de datos de escritura: RDW1.5.

Salida:

- Agente externo: usuario.
- Acción: confirmación de transferencia o error (saldo insuficiente o contraseña incorrecta).

RDE1.5: Datos de entrada de transferencia

- Nombre de usuario: cadena de caracteres (15)
- Cantidad: número real positivo con dos decimales
- Contraseña: cadena de entre 8 y 15 caracteres

RDR1.5: Datos leídos del perfil de usuario

- Nombre de usuario: cadena (15)
- cuenta_eliminada: booleano
- Saldo monedero: número real positivo con dos decimales
- Contraseña: cadena de entre 8 y 15 caracteres

RDW1.5: Datos de escritura en la BD

- Nombre de usuario: cadena (15)
- Nuevo saldo = saldo anterior – cantidad transferida (si saldo $\geq$ cantidad y contraseña correcta)

RF1.6: Dar de baja al usuario.

Entrada:

- Agente externo: usuario.
- Acción: solicitar eliminación de cuenta.
- Requisito de datos de entrada: RDE1.6

BD: Requisito de datos de escritura: RDW1.6.

Salida:

- Agente externo: usuario
- Acción: confirmación de baja o error (si hay ventas en curso, por ejemplo)

RDE1.6: Datos de entrada de baja

- Nombre de usuario: cadena (15)
- Contraseña: cadena de entre 8 y 15 caracteres

RDR1.6.1: Datos leídos del usuario.

- Nombre de usuario: cadena (15)
- cuenta_eliminada: booleano
- Contraseña: cadena de entre 8 y 15 caracteres

RDR1.6.2: Datos leídos del almacén de productos.

- Nombre de usuario: cadena (15)
- Id_producto de los productos ofertados por el usuario

RDR1.6.3: Datos leídos del almacén de ventas.

- Nombre de usuario: cadena (15)
- Id_producto de los productos ofertados por el usuario
- Booleano que indique si tiene ventas activas, es decir, si es vendedor o comprador de un producto vendido aún no confirmado, o si ha realizado alguna contraoferta que no ha sido aceptada ni rechazada.

RDW1.6: Datos de escritura en la BD

- Nombre de usuario: cadena (15)
- Se actualiza el campo cuenta_eliminada a *true*
- Se eliminan (vuelven a valores nulos o vacíos) todos los campos personales del usuario, excepto el nombre de usuario (clave primaria, se conserva)
- De los productos disponibles del usuario, se modifica su atributo disponible a *false*.

2.2. Gestión de productos (Roberto González)

RF2.1: Dar de alta producto

Entrada:

- Agente externo: usuario.
- Acción: solicitar inserción de producto.
- Requisito de datos de entrada: RDE2.1.

BD: Requisito de datos de escritura: RDW2.1.

Salida:

- Agente externo: usuario.
- Acción: confirmación de alta o mensaje de error.
- Requisito de datos de salida: RDS2.1

RDE2.1: Datos de entrada de alta de producto

- Título: cadena (máx. 80 caracteres)
- Descripción: cadena (máx. 500 caracteres)
- Precio: número real positivo con dos decimales (>0)
- Categoría: elemento de una jerarquía de categorías
- Imagen: 1 fichero .jpg de 1024x1024 píxeles

RDW2.1: Datos almacenados en la BD

- Id_producto: entero sin signo, generado automáticamente (único, no editable)
- Username_vendedor: referencia al usuario creador del producto
- Título: cadena (máx. 80 caracteres)
- Descripción: cadena (máx. 500 caracteres)
- Precio: número real positivo con dos decimales (>0)
- Categoría: elemento de jerarquía de categorías
- Imagen: 1 fichero .jpg de 1024x1024 píxeles
- Grado de promoción: número real entre 0 y 1 con dos decimales (por defecto 0)
- Disponible: bool que indica si el producto está disponible (por defecto true)

RDS2.1: Datos de salida

- Id_producto: identificador único del producto generado automáticamente

RF2.2: Modificar producto

Entrada:

- Agente externo: usuario.
- Acción: solicitar modificación de un producto propio.

- Requisito de datos de entrada: RDE2.2.

BD: Requisito de datos de escritura: RDW2.2.

Salida:

- Agente externo: usuario.
- Acción: confirmación de modificación o mensaje de error.

RDE2.2: Datos de entrada de modificación de producto

- Id_producto: entero sin signo (identificador del producto a modificar, obtenido al seleccionar el producto de forma visual en la aplicación)
- Título: cadena (máx. 80 caracteres, opcional)
- Descripción: cadena (máx. 500 caracteres, opcional)
- Precio: número real positivo con dos decimales (>0, opcional)
- Categoría: elemento de entre las disponibles categorías (opcional)
- Imágen: 1 fichero .jpg de 1024x1024 píxeles (opcional)

RDW2.2: Datos actualizados en la BD

- Título: cadena (máx. 80 caracteres)
- Descripción: cadena (máx. 500 caracteres)
- Precio: número real positivo con dos decimales (>0)
- Categoría: elemento de entre las disponibles categorías
- Imágen: 1 fichero .jpg de 1024x1024 píxeles
- Username_vendedor: referencia al usuario creador del producto (No se actualiza pero hace falta para realizar la operación)
- Disponible: bool que indica si el producto está disponible (por defecto true)

RF2.3: Eliminar producto

Entrada:

- Agente externo: usuario.
- Acción: solicitar la eliminación de un producto propio.
- Requisito de datos de entrada: RDE2.3.

BD: Requisito de datos de escritura: RDW2.3.

Salida:

- Agente externo: usuario.
- Acción: confirmación de eliminación o mensaje de error (por ejemplo, si el producto ya está en proceso de venta).

RDE2.3: Datos de entrada de eliminación de producto

- Id_producto: entero sin signo

RDW2.3: Datos modificados en la BD

- El estado “disponible” (pasa a false)

RF2.4: Consultar producto

Entrada:

- Agente externo: usuario.
- Acción: solicitar consulta de un producto.
- Requisito de datos de entrada: RDE2.4.

BD: Requisito de datos de lectura: RDR2.4.

Salida:

- Agente externo: usuario.
- Acción: mostrar información detallada del producto o mensaje de error si no existe.

RDE2.4: Datos de entrada de consulta de producto

- Id_producto: entero sin signo

RDR2.4: Datos leídos de la BD

- Id_producto: entero sin signo
- Username_vendedor: referencia al usuario creador del producto
- Título: cadena (máx. 80 caracteres)
- Descripción: cadena (máx. 500 caracteres)
- Precio: número real positivo con dos decimales (>0)
- Categoría
- Localización del vendedor
- Imágen: 1 fichero .jpg de 1024x1024 píxeles
- Grado de promoción: número real entre 0 y 1 con dos decimales
- Disponible: bool que indica si el producto está disponible (por defecto true)
- Valoración media del vendedor: número real entre 0 y 5 con un decimal (saltos de 0,5)

RDS2.4: Datos mostrados en pantalla

- Título
- Descripción
- Precio
- Categoría
- Localización
- Imágen

- Nombre de usuario del vendedor
- Valoración media del vendedor

RF2.5: Promocionar producto

Entrada:

- Agente externo: usuario.
- Acción: solicitar promoción de un producto propio.
- Requisito de datos de entrada: RDE2.5.

BD: Requisito de datos de escritura: RDW2.5.

Salida:

- Agente externo: usuario.
- Acción: confirmación de promoción o mensaje de error (saldo insuficiente o valor no válido).

RDE2.5: Datos de entrada de promoción

- Id_producto: entero sin signo
- Grado de promoción deseado: número real entre 0 y 1

RDW2.5: Datos almacenados en la BD

- Grado de promoción: número real entre 0 y 1 con dos decimales
- Disponible: bool que indica si el producto está disponible

Notas:

- El grado de promoción disminuye automáticamente en 0,1 cada 24 horas hasta alcanzar 0. (Esto sería un requisito funcional, pero por simplicidad de este ejercicio no se implementa).
- Un grado de promoción 1 implica visibilidad máxima durante 10 días.

2.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo)

RF3.1: Mostrar las recomendaciones

Entrada:

- Agente externo: usuario.
- Acción: abrir pestaña de recomendaciones.
- Requisito de datos de entrada RDE3.1

BD: Requisito de datos de lectura RDR3.1

Salida:

- Agente externo: usuario.
- Acción: confirmación de los resultados.
- Requisito de datos de salida: RDS3.1

RDE3.1:

- “username” del usuario que visualiza sus recomendaciones.

RDR3.1:

RDR3.1.1: RDE3.1, además de los siguientes:

- Categorías preferidas del usuario.
- Ubicación y rango de disponibilidad del usuario.
- Ubicación y rango de disponibilidad de los vendedores.
- Puntuación de los perfiles de vendedores.
- username de los vendedores

RDR3.1.2:

- id_producto de los recomendados
- Disponibilidad de los productos
- Categorías preferidas del usuario buscador (para filtrado)
- Categorías de los productos recomendados
- promoción de los productos recomendados
- Título de los susodichos productos
- Imagen de los susodichos productos
- Precio de los susodichos productos
- número de favoritos de los productos

RDR3.1.3:

- username de los vendedores
- id_producto de los recomendados

RDS3.1: Listado de productos que cumplan con las restricciones de rango y categoría mencionadas en RDR3.1 y ordenados según el criterio detallado en la sección “Feed de recomendaciones, búsquedas y favoritos”, mostrando para cada producto:

- Título de producto.

- Imagen del producto.
- Precio del producto.

RF3.2: Marcar un producto como favorito

Entrada:

- Agente externo: usuario.
- Acción: visualizando un producto, pulsar el botón de “Añadir a favoritos”.
- Requisito de datos de entrada RDE3.2

BD: Requisito de datos de escritura RDW3.2 y requisito de datos de lectura RDR3.2

Salida:

- Agente externo: usuario.
- Acción: confirmación resultado.
- Requisito de datos de salida: ninguno.

RDE3.2:

- “Nombre de usuario” del usuario que añade un producto a sus favoritos.
- id_producto del producto que se añade a favoritos.

RDW3.2: RDE3.2, además de

- Lista de id_producto, los “favoritos” del usuario.

RDR3.2: RDE3.2, además de

- Disponibilidad del producto

RF3.3: Quitar un producto de “favoritos”

Entrada:

- Agente externo: usuario.
- Acción: visualizando un producto, pulsar el botón de “Quitar de favoritos”.
- Requisito de datos de entrada RDE3.3

BD: Requisito de datos de escritura RDW3.2

Salida:

- Agente externo: usuario.
- Acción: confirmación resultado.
- Requisito de datos de salida: ninguno.

RDE3.3: Los mismo que RDE3.2

RDW3.3: Los mismos que en RDW3.2

RF3.4: Consultar los favoritos

Entrada:

- Agente externo: usuario.
- Acción: abrir la pestaña de “Favoritos”.
- Requisito de datos de entrada RDE3.4

BD: Requisito de datos de lectura RDR3.4

Salida:

- Agente externo: usuario.
- Acción: confirmación de resultados.
- Requisitos de datos de salida: RDS3.4

RDE3.4:

- Nombre de usuario del usuario que consulta sus favoritos.

RDR3.4:

RDR3.4.1:

- Lista de id_producto, los favoritos del usuario.

RDR3.4.2:

- id_producto de los favoritos.
- Título de producto.
- Imagen del producto.
- Precio del producto.
- Disponibilidad

RDS3.4: Listado de productos favoritos de un usuario, mostrando para cada uno los campos mencionados en RDR3.4

RF3.5: Hacer una búsqueda

Entrada:

- Agente externo: usuario.
- Acción: pulsar el botón de buscar tras seleccionar opcionalmente una categoría de búsqueda y una cadena de caracteres además de si desea ordenar por precio, pudiendo hacerlo ascendente o descendientemente.
- Requisito de datos de entrada RDE3.5

BD: Requisito de datos de lectura RDR3.5

Salida:

- Agente externo: usuario
- Acción: ninguna
- Requisito de datos de salida: RDS3.5

RDE3.5:

- “Nombre de usuario” del usuario que hace la búsqueda
- Categoría de filtrado
- Nombre de filtrado de productos
- Ordenación de precio ascendente o descendente (opcional)

RDR3.5:**RDR3.5.1:**

- username del usuario que hace la búsqueda
- Ubicación y rango de disponibilidad del usuario.
- Ubicación y rango de disponibilidad de los vendedores.
- Puntuación de los perfiles de vendedores.
- username de los vendedores

RDR3.5.2:

- id_producto de los productos buscados
- Categoría de filtrado
- Nombre de filtrado
- Si están disponibles los productos
- Categorías de los productos buscados
- promoción de los productos buscados
- Título de los susodichos productos
- Imagen de los susodichos productos
- Precio de los susodichos productos

RDR3.5.3:

- username de los vendedores
- id_producto de los productos buscados

RDS3.5: Listado de productos que cumplan con las restricciones de rango, categoría y título mencionadas en RDR3.5 ordenados según el criterio explicado en la sección de “Feed de recomendaciones, búsquedas y favoritos”, mostrando para cada producto:

- Título de producto.
- Imagen del producto.
- Precio del producto.
- Categoría
- promoción
- nombre del vendedor
- valoración del vendedor

2.4. Gestión de ventas (Juan Manuel Fernández)

RF4.1: Realizar compra directa de producto.

Entrada:

- Agente externo: Comprador.
- Acción: Retirar dinero del monedero del comprador.
- Requisito de datos entrada RDE4.1

BD: Requisito de datos de escritura RDW4.1

Salida:

- Agente externo: No
- Acción: Confirmación de la compra.
- Requisito de datos de salida: Ninguno.

RDE4.1: Datos de entrada de la compra:

- Identificador de producto: unsigned int
- Identificador de comprador: Cadena de caracteres (15)
- Identificador de vendedor: Cadena de caracteres (15)

RDW4.1: Datos almacenados del comprador y vendedor.

- Precio de producto: Número real no negativo con dos cifras decimales.
- Monedero del comprador: Contador del saldo del comprador.
- Estado de recepción del producto: booleano indicando si el producto está pendiente de ser recibido.

RF4.2: Realizar contraoferta de producto.

Entrada:

- Agente externo: Comprador.
- Acción: Añadir contraoferta al producto.
- Requisito de datos de entrada RDE4.2

BD: Requisito de datos de escritura RDW4.2

Salida:

- Agente externo: No.
- Acción: Notificar contraoferta al vendedor y confirmar éxito de la operación al comprador.
- Requisito de datos de salida: Ninguno.

RDE4.2: Datos de contraoferta:

- Identificador de producto: unsigned int

- Identificador de comprador: Cadena de caracteres (15)
- Identificador de vendedor: Cadena de caracteres (15)
- Precio contraoferta: Número real no negativo con dos cifras decimales.

RDW4.2: Datos almacenados de contraofertas asociadas al producto.

- Los mismos que RDE4.2

RF4.3: Aceptar/Rechazar contraoferta de producto.

Entrada:

- Agente externo: Vendedor
- Acción: Aceptar o rechazar la contraoferta
- Requisito de datos de entrada RDE4.3

BD: Requisito de datos de escritura RDW4.3 en caso de aceptar.

Salida:

- Agente externo: No
- Acción: Notificar al comprador sobre el estado de la contraoferta y al vendedor del éxito de la operación.
- Requisito de datos de salida: Ninguno

RDE4.3: Datos de aceptación de la contraoferta

- Aceptación de la contraoferta: booleano

RDW4.3: Datos de contraoferta:

- Los mismos que RDE4.2

RF4.4: Consultar contraofertas asociadas a un producto.

Entrada:

- Agente externo: Usuario
- Acción: Consulta de contraofertas asociadas a un producto.
- Requisito de datos de entrada RDE4.4

BD: Requisito de datos de lectura RDR4.4

Salida:

- Agente externo: No
- Acción: Mostrar contraofertas asociadas a un producto.
- Requisito de datos de salida RDS4.4

RDE4.4: Datos de identificación del producto.

- Identificador de producto: unsigned int

RDR4.4: Datos de contraofertas asociadas al producto.

Para cada contraoferta asociada al producto:

- Identificador del comprador: Cadena de caracteres (15)
- Precio contraoferta: Número real no negativo con dos cifras decimales.

RDS4.4: Lista de contraofertas asociadas al producto.

Para cada contraoferta asociada al producto:

- Identificador contraoferta: Cadena de caracteres (15)
- Precio contraoferta: Número real no negativo con dos cifras decimales.

RF4.5: Confirmación compra del producto.

Entrada:

- Agente externo: No
- Acción: Ninguna
- Requisito de datos de entrada: Ninguno

BD: Requisito de datos de escritura RDW.5

Salida:

- Agente externo: No
- Acción: Confirmar compra del producto
- Requisito de datos de salida: RDS4.5

RDW4.5: Estado de recepción del producto:

- Estado de recepción del producto: booleano indicando si el producto está pendiente de ser recibido.

RDS4.5: Confirmación de compra del producto.

- Estado del producto: booleano indicando si se ha realizado la compra con éxito.

RF4.6: Puntuar compra del producto.

Entrada:

- Agente externo: Comprador
- Acción: Puntuar compra de producto
- Requisito de datos de entrada RDE4.6

BD: Requisito de datos de escritura RDW4.6

Salida:

- Agente externo: No
- Acción: Ninguna
- Requisito de datos de salida: Ninguno

RDE4.6: Datos de compra del producto

- Identificador del producto: unsigned int
- Puntuación del producto: Número real

RDW4.6: Puntuación media del usuario

- Identificador del usuario: Cadena de caracteres (15)
- Identificador del producto: unsigned int
- Puntuación recibida: Número real perteneciente al conjunto {0,0.5,1,1.5,2,2.5,3,3.5,4,4.5,5}

2.5. Gestión de mensajes (Aitor de la Iglesia)

RF5.1: Enviar mensaje

Entrada:

- Agente externo: Usuario
- Acción: Escribir el mensaje
- Requisito de datos de entrada RDE5.1

BD: Requisito de datos de escritura RDW5.1, RDR5.1

Salida:

- Agente externo: Usuario
- Acción: Mostrar el mensaje enviado y notificar por correo al receptor
- Requisito de datos de salida RDS5.1.1, RDS5.1.2

RDE5.1: Datos de mensaje a enviar

- Mensaje: cadena de caracteres (200)
- Id_vendedor: cadena de caracteres (15)
- Id_comprador: cadena de caracteres (15)
- Id_producto: unsigned int
- Fecha de envío: tipoDate
- Emisor: cadena de caracteres (15)

RDW5.1: Datos almacenados en la conversación

- Los mismos que RDE5.1
- Leído: bool

RDR5.1: Correo receptor:

- Id_receptor: cadena de caracteres (15)
- correo_receptor: cadena de caracteres formato estándar

RDS5.1.1: Último mensaje

- Id_vendedor: cadena de caracteres (15)
- Id_comprador: cadena de caracteres (15)
- Id_producto: unsigned int
- Mensaje: cadena de caracteres (200)
- Fecha de envío: tipoDate

RDS5.1.2: Notificación

- Correo_receptor: cadena de caracteres
- Lo mismo que RDS5.1.1

RF5.2: Consultar conversación

Entrada:

- Agente externo: Usuario
- Acción: Seleccionar una conversación
- Requisito de datos de entrada RDE5.2

BD: Requisito de datos de lectura RDR5.2, RDW5.2

Salida:

- Agente externo: Usuario
- Acción: Mostrar mensajes de la conversación
- Requisito de datos de salida RDS5.2

RDE5.2: Conversación a seleccionar

- id_comprador: cadena de caracteres (15)
- id_vendedor: cadena de caracteres (15)
- Id_producto: unsigned int

RDR5.2: Mensajes almacenados de la conversación

- Id_vendedor: cadena de caracteres (15)
- Id_comprador: cadena de caracteres (15)
- Id_producto: unsigned int
- Emisor: cadena de caracteres (15)
- Mensajes: cadena de caracteres (200)
- Archivo: archivo
- Fecha de envío: tipo Date
- Leído: bool
- Archivado: bool

RDW5.2:

- Id_vendedor: cadena de caracteres (15)
- Id_comprador: cadena de caracteres (15)
- Id_producto: unsigned int
- Leído: bool

RDS5.2: Mensajes de la conversación

- Lo mismo que RDR5.2 pero formateado

RF5.3: Adjuntar archivo

Entrada:

- Agente externo: Usuario
- Acción: Pulsar adjuntar
- Requisito de datos de entrada RDE5.3

BD: Requisito de datos de escritura RDW5.3

Salida:

- Agente externo: Usuario
- Acción: Mostrar el mensaje enviado y notificar por correo al receptor
- Requisito de datos de salida RDS5.3

RDE5.3: Archivo a adjuntar

- id_comprador: cadena de caracteres (15)
- id_vendedor: cadena de caracteres (15)
- Id_producto: unsigned int
- Emisor: cadena de caracteres (15)
- Archivo: archivo
- Fecha : tipo Date

RDW5.3: Archivo almacenado en la conversación

- Los mismos datos que RDE5.3
- Leído: bool

RDS5.3: Archivo enviado

- Lo mismo que RDE5.3

RF5.4: Buscar mensaje

Entrada:

- Agente externo: Usuario
- Acción: Seleccionar unos filtros
- Requisito de datos de entrada RDE5.4

BD: Requisito de datos de lectura RDR5.4

Salida:

- Agente externo: Usuario
- Acción: Mostrar mensajes concordantes
- Requisito de datos de salida RDS5.4

RDE5.4: Mensajes a buscar

- Palabras clave: cadena de caracteres (200)
- Usuario: cadena de caracteres (15)
- Fecha: tipo Date

RDR5.4: Mensajes almacenados de la conversación

- Palabras clave: cadena de caracteres (200)
- Id_vendedor: cadena de caracteres (15)

- Id_comprador: cadena de caracteres (15)
- Id_producto: unsigned int
- Emisor: cadena de caracteres (15)
- Mensajes: cadena de caracteres (200)
- Archivo: archivo
- Fecha de envío: tipo Date
- Leído: bool
- Archivado: bool

RDS5.4: Mensajes de la conversación

- Subconjunto de RDR5.4 coincidente con RDE5.4 y formateado

RF5.5: Archivar conversación

Entrada:

- Agente externo: SI
- Acción: Una compra ha sido completada
- Requisito de datos de entrada RDE5.5

BD: Requisito de datos de lectura RDW5.5

Salida:

RDE5.5: Conversación a archivar

- id_comprador: cadena de caracteres (15)
- id_vendedor: cadena de caracteres (15)
- Id_producto: unsigned int

RDW5.5: Conversación a archivar almacenada

- id_comprador: cadena de caracteres (15)
- id_vendedor: cadena de caracteres (15)
- Id_producto: unsigned int
- Archivado: bool

3. Restricciones semánticas

3.1. Gestión de perfiles (Elsa Rodríguez)

RS1.1:

Un usuario debe ser mayor de edad para registrarse en la plataforma.

RF: RF1.1.

RD: RDE1.1.

Descripción: Si el usuario no confirma su mayoría de edad, no se insertan los datos del nuevo usuario y se devuelve un error.

RS1.2:

El nombre de usuario y el correo electrónico deben ser únicos en el sistema.

RF: RF1.1.

RD: RDR1.1, RDW1.1.

Descripción: Si el nombre de usuario o el correo electrónico ya existen en la base de datos, no se realiza la inserción y se devuelve un mensaje de error.

RS1.3:

El nombre de usuario y el correo electrónico deben ser únicos en el sistema.

RF: RF1.3.

RD: RDR1.3, RDW1.3.

Descripción: Si el nombre de usuario o el correo electrónico ya existen en la base de datos, no se realiza la inserción y se devuelve un mensaje de error.

RS1.4:

La contraseña debe cumplir con los requisitos de complejidad.

RF: RF1.1.

RD: RDE1.1.

Descripción: Si la contraseña no tiene entre 8 y 15 caracteres, o no contiene al menos una mayúscula, una minúscula, un número y un carácter especial (sin espacios), no se permite la inserción o modificación.

RS1.5:

El usuario debe aceptar la política de privacidad y las condiciones de uso.

RF: RF1.1.

RD: RDE1.1.

Descripción: Si el campo de aceptación no es verdadero, no se completa el registro y se devuelve un error.

RS1.6:

Un usuario no puede tener menos de una ni más de seis categorías de preferencia.

RF: RF1.1.

RD: RDE1.1.

Descripción: Si el número de categorías seleccionadas está fuera del rango [1,6], no se guardan los datos y se devuelve un error.

RS1.7:

Un usuario no puede tener menos de una ni más de seis categorías de preferencia.

RF: RF1.3.

RD: RDE1.3.

Descripción: Si el número de categorías seleccionadas está fuera del rango [1,6], no se guardan los datos y se devuelve un error.

RS1.8:

La cantidad a añadir al monedero debe ser positiva.

RF: RF1.4.

RD: RDE1.4.

Descripción: Si la cantidad es menor o igual que cero, no se realiza la operación y se devuelve un error.

RS1.9:

La cantidad a transferir a una cuenta bancaria debe estar comprendida entre 0 y el saldo disponible.

RF: RF1.5.

RD: RDE1.5, RDR1.5.

Descripción: Si la cantidad es menor o igual que cero, o superior al saldo actual del usuario, no se realiza la transferencia y se devuelve un error.

RS1.10:

No se puede transferir saldo si la contraseña es incorrecta.

RF: RF1.5.

RD: RDE1.5.

Descripción: Si la contraseña introducida no coincide con la registrada, la operación se cancela y se devuelve un error de autenticación.

RS1.11:

Un usuario no puede eliminar su cuenta si tiene ventas activas en curso.

RF: RF1.6.

RD: RDE1.6, RDW1.6.

Descripción: Si existen registros de ventas no finalizadas asociados al usuario, no se realiza la baja y se devuelve un aviso.

RS1.12:

Ninguna operación de gestión de perfil podrá ejecutarse si el usuario no existe.

RF: RF1.2.

RD: RDR1.2.

Descripción: Si el campo `cuenta_eliminada` del usuario es verdadero, no se realiza la operación y se devuelve un error indicando que la cuenta ha sido eliminada.

RS1.13:

Ninguna operación de gestión de perfil podrá ejecutarse si el usuario no existe.

RF: RF1.3.

RD: RDR1.3.

Descripción: Si el campo `cuenta_eliminada` del usuario es verdadero, no se realiza la operación y se devuelve un error indicando que la cuenta ha sido eliminada.

RS1.14:

Ninguna operación de gestión de perfil podrá ejecutarse si el usuario no existe.

RF: RF1.4.

RD: RDR1.4.

Descripción: Si el campo `cuenta_eliminada` del usuario es verdadero, no se realiza la operación y se devuelve un error indicando que la cuenta ha sido eliminada.

RS1.15:

Ninguna operación de gestión de perfil podrá ejecutarse si el usuario no existe.

RF: RF1.5.

RD: RDR1.5.

Descripción: Si el campo `cuenta_eliminada` del usuario es verdadero, no se realiza la operación y se devuelve un error indicando que la cuenta ha sido eliminada.

RS1.16:

Ninguna operación de gestión de perfil podrá ejecutarse si el usuario no existe.

RF: RF1.6.

RD: RDR1.6.

Descripción: Si el campo `cuenta_eliminada` del usuario es verdadero, no se realiza la operación y se devuelve un error indicando que la cuenta ha sido eliminada.

RS1.17:

El rango de interés debe ser positivo.

RF: RF1.1.

RD: RDE1.1.

Descripción: Si la cantidad es menor o igual que cero, no se realiza la inserción y se devuelve un error.

RS1.18:

El rango de interés debe ser positivo.

RF: RF1.3.

RD: RDE1.3.

Descripción: Si la cantidad es menor o igual que cero, no se realiza la modificación y se devuelve un error.

3.2. Gestión de productos (Roberto González)

RS2.1:

El **precio** de un producto debe ser **mayor que 0** y expresarse con **dos decimales**.

RF: RF2.1

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si el precio es ≤ 0 o no tiene exactamente dos decimales, se rechaza la operación y se devuelve un error.

RS2.2:

El **precio** de un producto debe ser **mayor que 0** y expresarse con **dos decimales**.

RF: RF2.2.

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si el precio es ≤ 0 o no tiene exactamente dos decimales, se rechaza la operación y se devuelve un error.

RS2.3:

La **categoría** elegida debe pertenecer a la **lista de categorías** vigente.

RF: RF2.1.

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si la categoría no existe en el sistema, se cancela la operación y se informa del error.

RS2.4:

La **categoría** elegida debe pertenecer a la **lista de categorías** vigente

RF: RF2.2.

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si la categoría no existe en el sistema, se cancela la operación y se informa del error.

RS2.5:

El **título** y la **descripción** deben respetar las longitudes máximas: **título ≤ 80 y descripción ≤ 500** caracteres.

RF: RF2.1.

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si se exceden los límites, se rechaza la operación y se devuelve un error.

RS2.6:

El **título** y la **descripción** deben respetar las longitudes máximas: **título ≤ 80 y descripción ≤ 500** caracteres.

RF: RF2.2.

RD: RDE2.1, RDE2.2, RDW2.1, RDW2.2.

Descripción: Si se exceden los límites, se rechaza la operación y se devuelve un error.

RS2.7:

Solo el **propietario (Username_vendedor)** puede **promocionar** su producto.

RF: RF2.5.

RD: RDE2.2, RDE2.3, RDE2.5, RDW2.2, RDW2.3, RDW2.5.

Descripción: Si el usuario autenticado no coincide con el propietario del producto, la operación se deniega y se devuelve un error de autorización.

RS2.8:

Solo se puede **modificar** un producto que esté disponible y no tenga contraofertas activas

RF: RF2.2.

RD: RDE2.2, RDE2.3, RDW2.2, RDW2.3.

Descripción: Si el producto está en venta, se cancela la operación y se devuelve un mensaje informativo.

RS2.9:

Solo se puede **eliminar** un producto que esté disponible y no tenga contraofertas activas

RF: RF2.3.

RD: RDE2.2, RDE2.3, RDW2.2, RDW2.3.

Descripción: Si el producto tiene contraofertas o está pendiente de confirmación, se cancela la operación y se devuelve un mensaje informativo.

RS2.10:

El **grado de promoción** debe estar en el intervalo **[0, 1]** con **dos decimales**.

RF: RF2.5.

RD: RDE2.5, RDW2.5.

Descripción: Si el valor solicitado está fuera de rango o no cumple el formato de dos decimales, la promoción se rechaza y se devuelve un error.

RS2.11:

El **decaimiento automático** del grado de promoción es de **0,1 cada 24 horas** y **no puede resultar en valores negativos** (se trunca a 0 en caso de ser ≤ 0).

RF: RF2.5.

RD: RDW2.5.

Descripción: El sistema aplica el decremento periódico desde la "Fecha/hora de inicio de promoción"; si el cálculo resultara < 0 , se fija en 0.

RS2.12:

El **Id_producto** se **genera automáticamente**, es **único** y **no editable** por el usuario.

RF: RF2.1.

RD: RDW2.1, RDW2.2, RDR2.4.

Descripción: Cualquier intento de fijar o alterar manualmente el identificador es ignorado y/o genera un error.

RS2.13:

El **Id_producto** se **genera automáticamente**, es **único** y **no editable** por el usuario.

RF: RF2.2.

RD: RDW2.1, RDW2.2, RDR2.4.

Descripción: Cualquier intento de fijar o alterar manualmente el identificador es ignorado y/o genera un error.

RS2.14:

El **Username_vendedor** debe corresponder a un **usuario existente**.

RF: RF2.1.

RD: RDW2.1.

Descripción: Si el vendedor no existe o su cuenta está marcada como eliminada, se rechaza el alta del producto.

RS2.15:

La **valoración media de un vendedor** se muestra con un decimal (ej. 4.5) y **no puede modificarse**.

RF: RF2.4.

RD: RDR2.4, RDS2.4.

Descripción: La valoración mostrada no se edita en esta operación; cualquier modificación de valoraciones se gestiona en el subsistema correspondiente.

3.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo)

RS3.1:

Solo deben aparecer en el feed productos que estén disponibles.

RF: RF3.1

RD(s): RDR3.1.2

Descripción: No tiene sentido que los productos no disponibles para venta se muestren a un usuario que podría querer comprarlos.

RS3.2:

Solo deben poder consultarse como favoritos productos que estén disponibles.

RF: RF3.4

RD(s): RDR3.4.2

Descripción: No tiene sentido que los productos no disponibles para venta se muestren a un usuario en sus favoritos, porque ahí solo guarda aquellos productos que podría querer comprar.

RS3.3:

Solo deben aparecer en las búsquedas productos que estén disponibles

RF: RF3.5

RD(s): RDR3.5.2

Descripción: No tiene sentido que los productos no disponibles para venta se muestren a un usuario en sus una búsqueda orientada a compra.

3.4. Gestión de ventas (Juan Manuel Fernández)

RS4.1:

Una compra puede ser realizada sólo si se dispone de saldo suficiente en el monedero.

RF: RF4.1

RD(s): RDW4.1

Descripción: Si no se dispone de saldo suficiente en el monedero para hacer la compra, se cancela la operación y se notifica al comprador de que no es posible realizar la compra.

RS4.2:

El precio de una contraoferta debe ser positivo y menor estricto que el precio del producto estipulado por el vendedor.

RF: RF4.2

RD(s): RDW4.2

Descripción: Si el precio de la contraoferta no cumple con las restricciones anteriormente mencionadas, se cancela la operación y se informa al comprador de que su contraoferta no es válida.

RS4.3:

No se pueden realizar contraofertas superiores al saldo actual del monedero.

RF: RF4.2

RD(s): RDW4.2

Descripción: Si el precio de la contraoferta es superior al saldo actual del monedero, se cancela la operación y se informa al comprador de que su contraoferta no es viable.

RS4.4:

Un usuario sólo puede consultar contraofertas asociadas a sus productos en venta.

RF: RF4.4

RD(s): RDR4.4

Descripción: Un usuario puede consultar únicamente las contraofertas que están asociadas a los productos que haya puesto en venta.

RS4.5

El estado del producto debe ser comprado, pendiente de compra o en venta.

RF: RF4.5

RD(s): RDW4.5

Descripción: El estado del producto debe ser en todo momento comprado, pendiente de compra o en venta.

RS4.6:

La puntuación de una compra debe ser un valor perteneciente al conjunto {0,0.5,1,1.5,2,2.5,3,3.5,4,4.5,5}.

RF: RF4.6

RD(s): RDW4.6

Descripción: La puntuación de una compra debe ser un valor real perteneciente al conjunto {0,0.5,1,1.5,2,2.5,3,3.5,4,4.5,5}.

RS4.7:

Un usuario no puede comprarse productos a sí mismos ni realizar contraofertas por sus propios productos.

RF: RF4.1, RF4.2

RD(s): RDE4.1, RDE4.2

Descripción: Los usuarios sólo pueden comprar y realizar contraofertas a productos que pertenezcan a otros usuarios.

3.5. Gestión de mensajes (Aitor de la Iglesia)

RS5.1:

Los usuarios que formen parte de una conversación activa y el producto por el que negocian deben estar registrados en la aplicación y estar activos..

RF: RF5.1

RD: RDE5.1, RDW5.1

Descripción: el usuario no podrá enviar un mensaje si no se cumple la condición

RS5.2:

Los usuarios que formen parte de una conversación activa y el producto por el que negocian deben estar registrados en la aplicación y estar activos..

RF: RF5.3

RD: RDE5.3, RDR5.3, RDW5.3

Descripción: el usuario no podrá enviar un mensaje si no se cumple la condición

RS5.3:

Los usuarios que forman parte de una conversación activa deben tener una venta en proceso:

RF: RF5.1

RD: RDE5.1, RDW5.1

Descripción: el usuario no podrá enviar un mensaje si no se cumple la condición

RS5.4:

Una conversación no puede estar activa y archivada a la vez

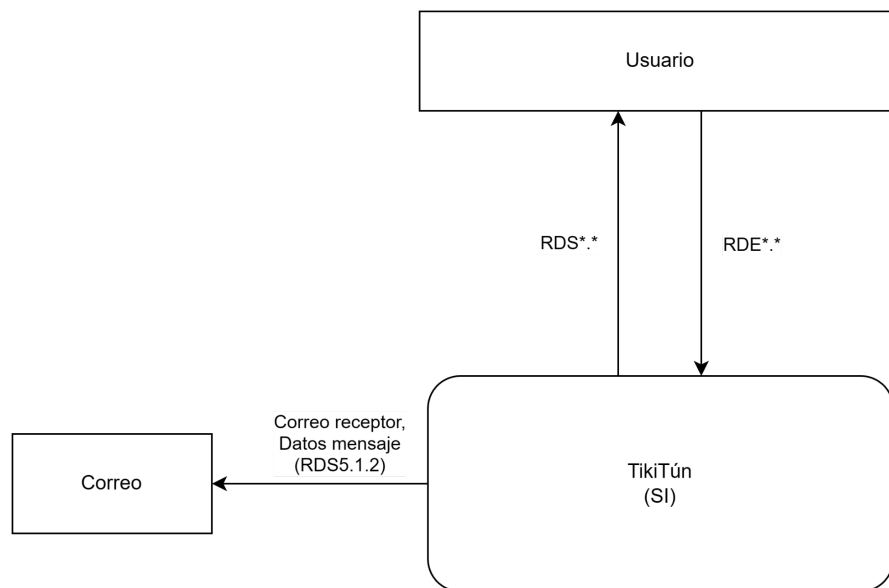
RF: RF5.5

RD: RDW5.5

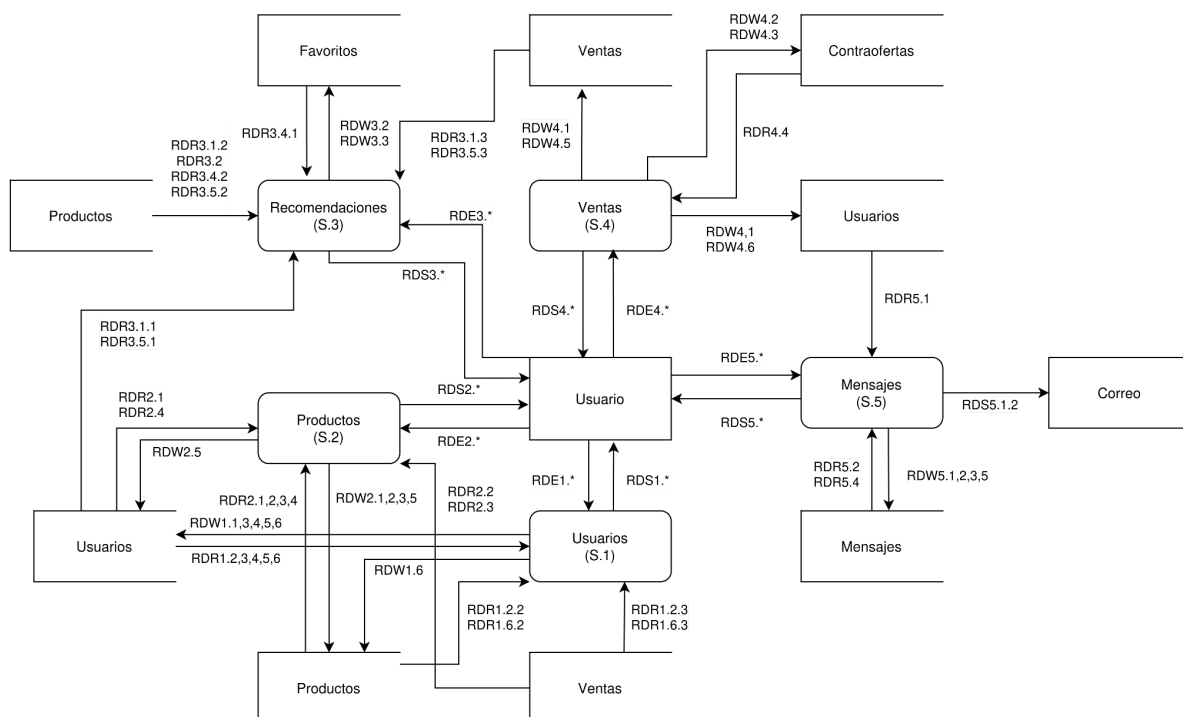
Descripción: el usuario no podrá enviar un mensaje si no se cumple la condición

4. Diagramas y paso a tablas

4.1. Caja Negra

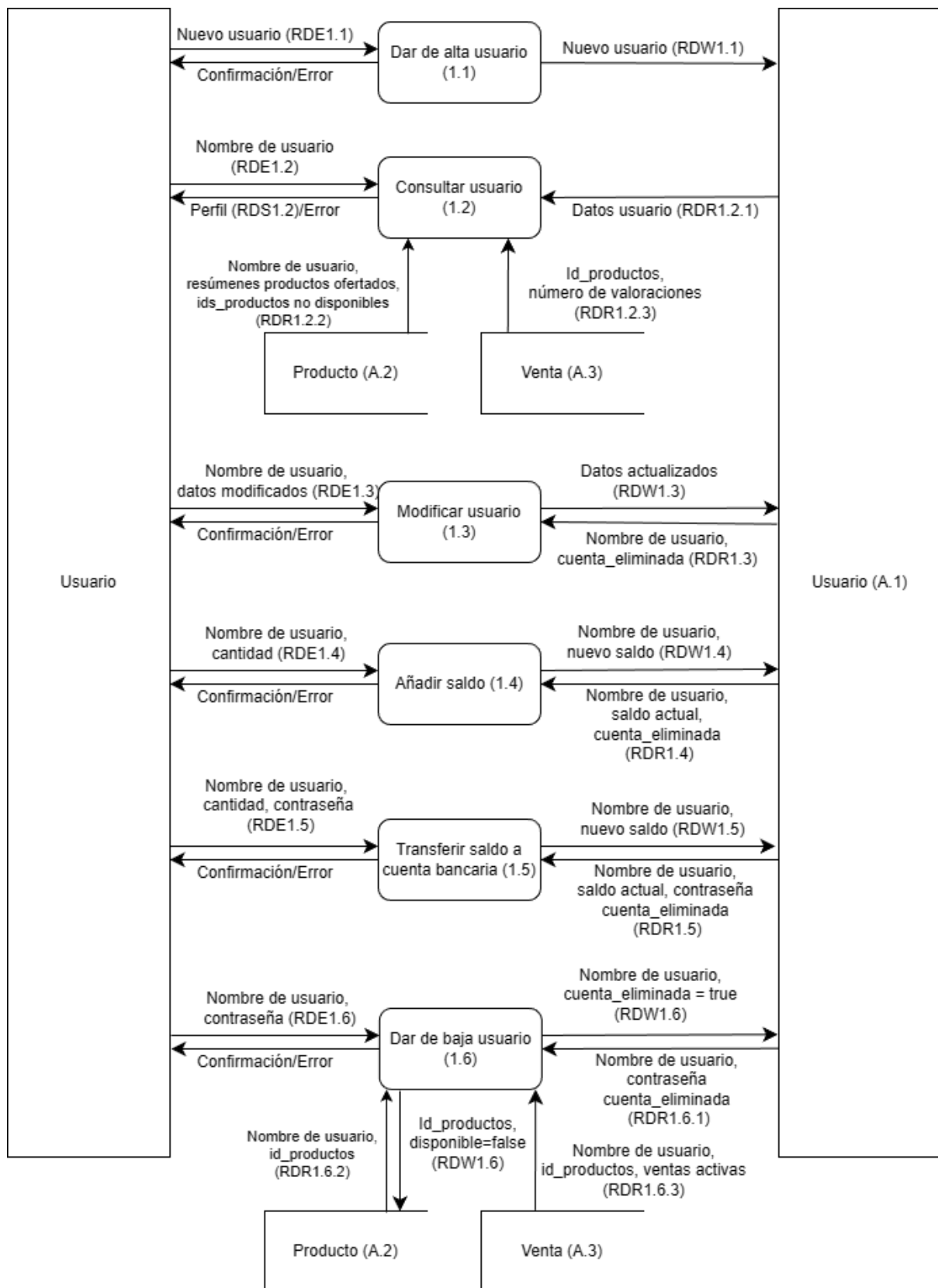


4.2. Armazón

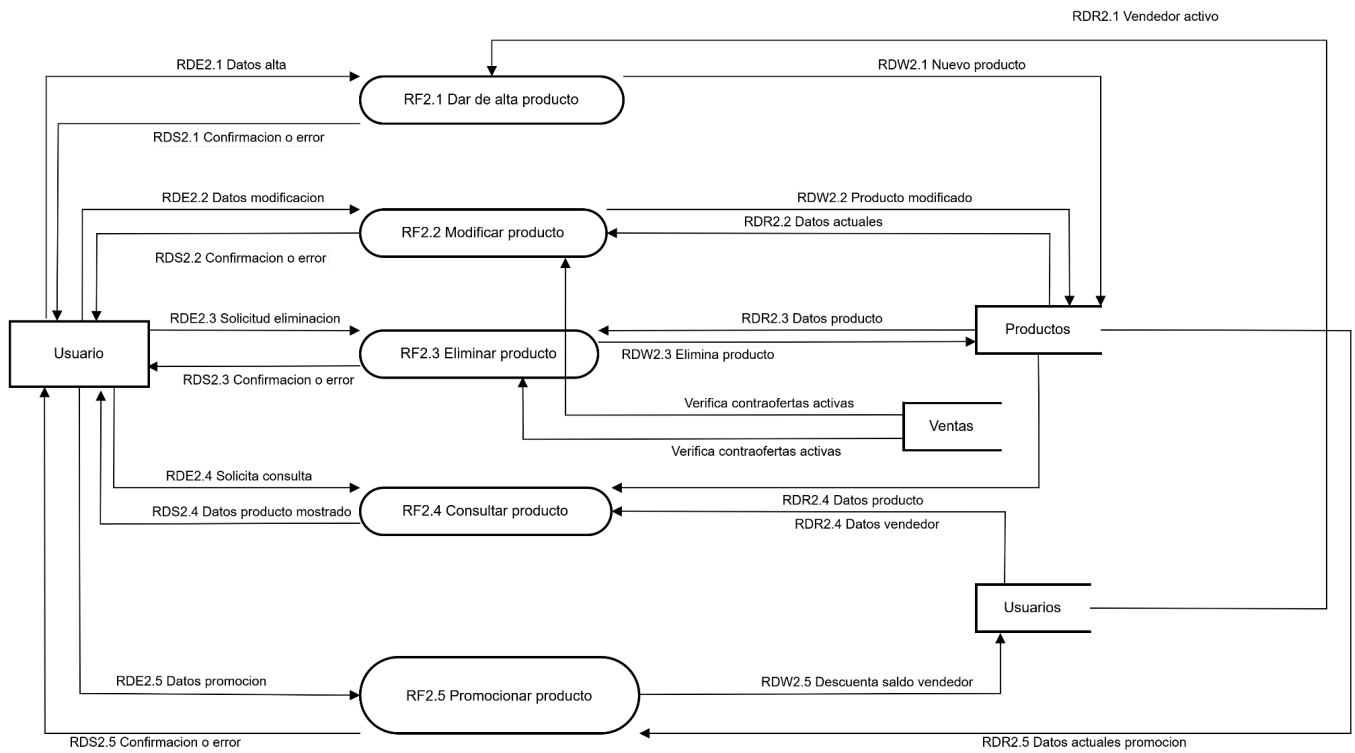


4.3. DFD1s

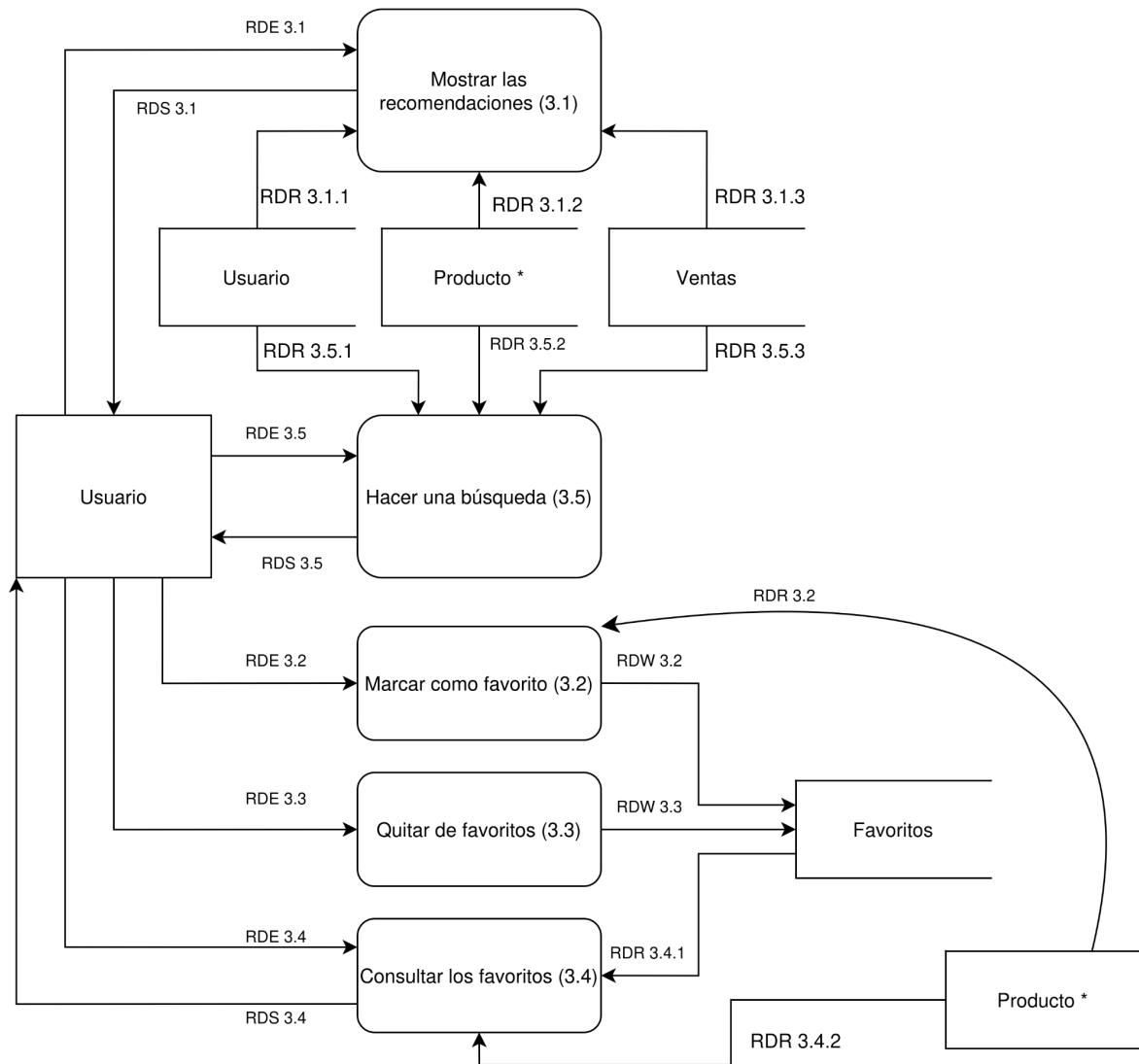
4.3.1. Gestión de Perfiles (Elsa Rodríguez)



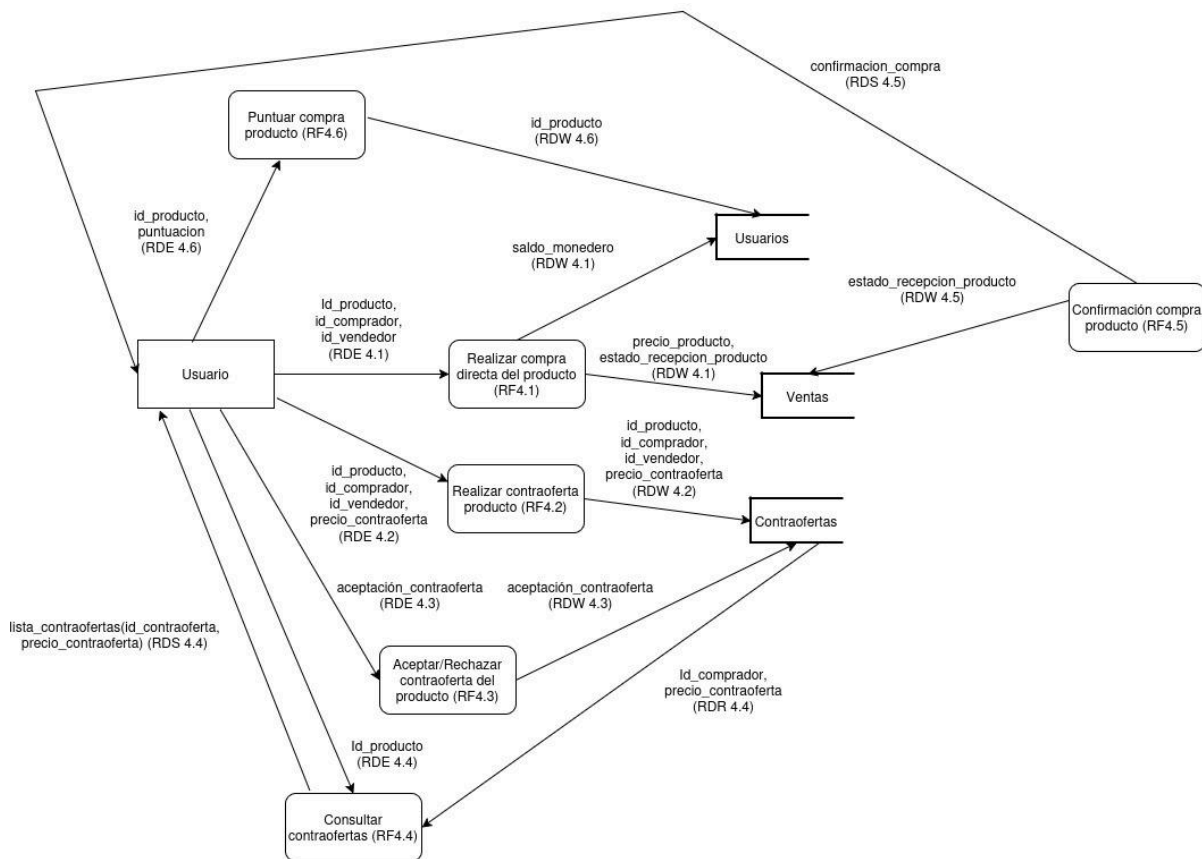
4.3.2. Gestión de productos (Roberto González)



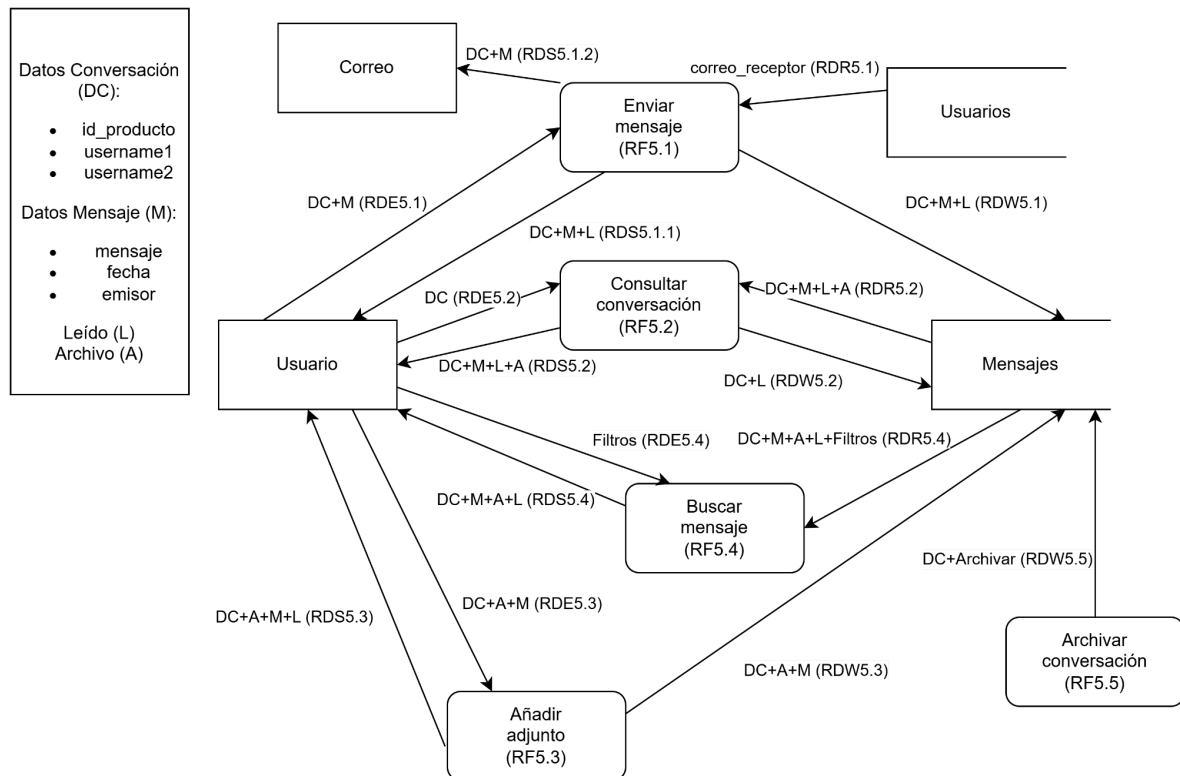
4.3.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo)



4.3.4. Gestión de ventas (Juan Manuel Fernández)

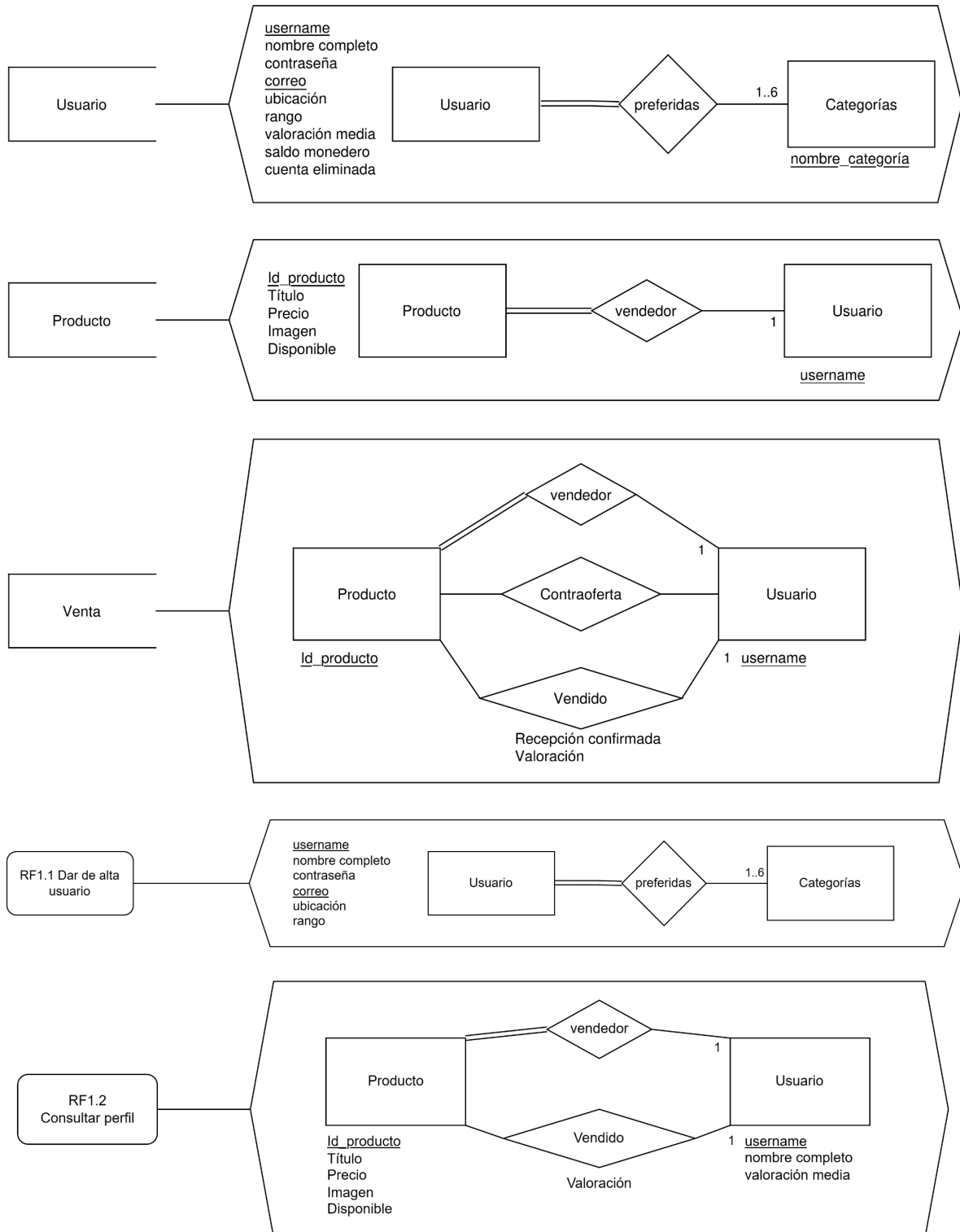


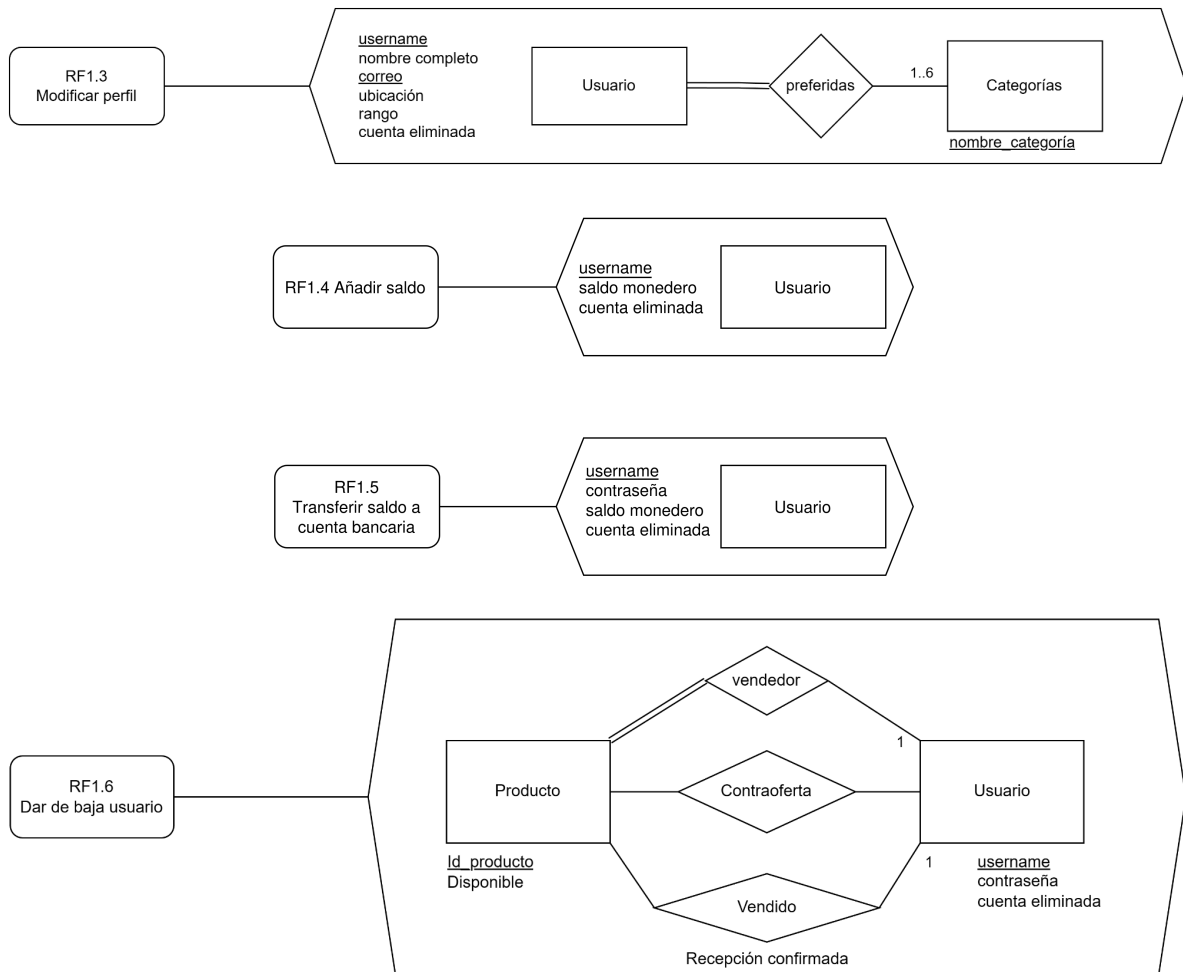
4.3.5. Gestión de mensajes (Aitor de la Iglesia)



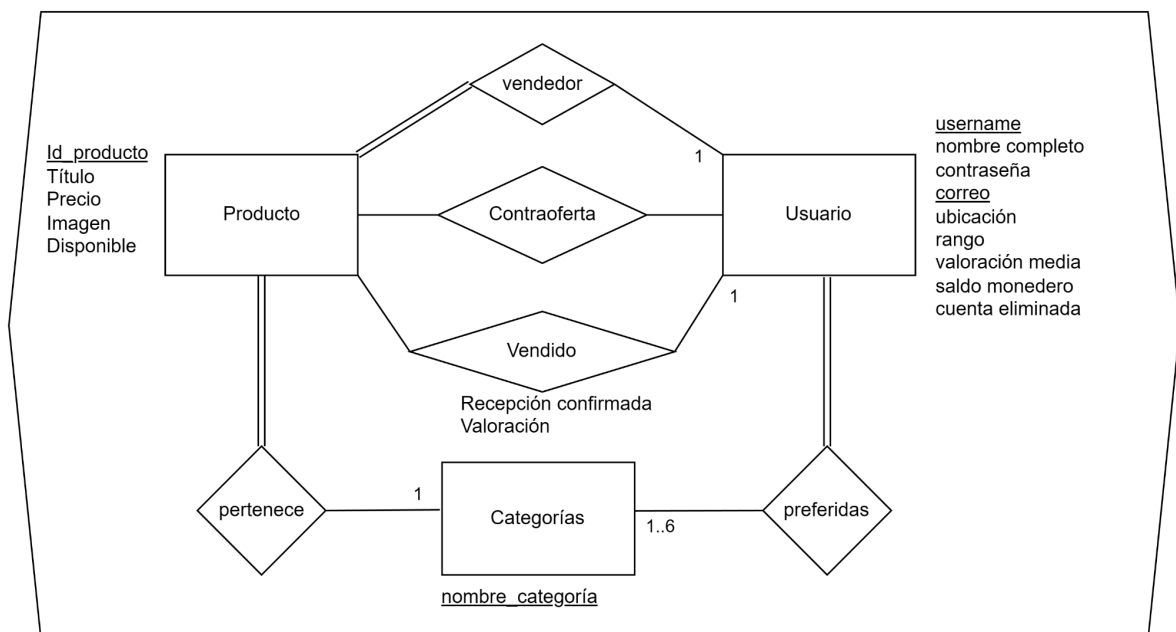
4.4. Esquemas externos

4.4.1. Gestión de Perfiles (Elsa Rodríguez)

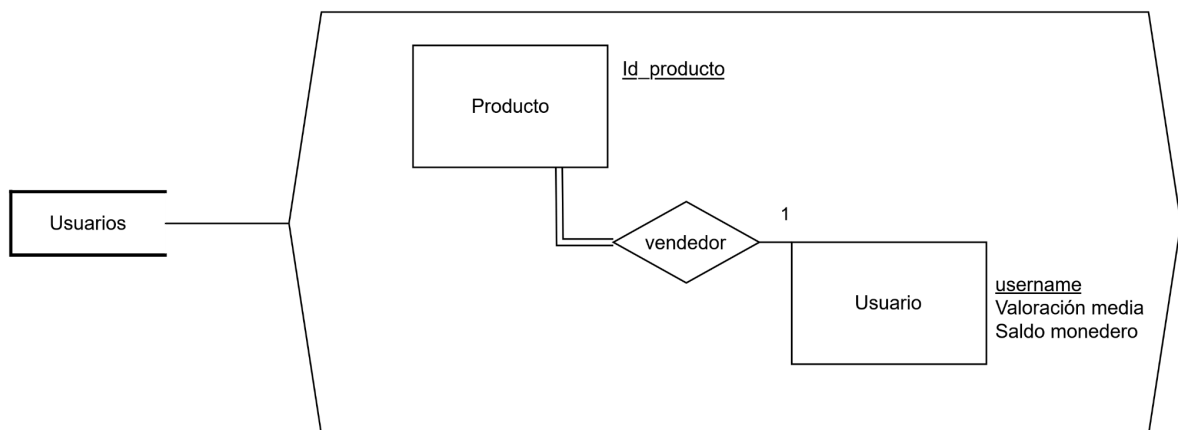
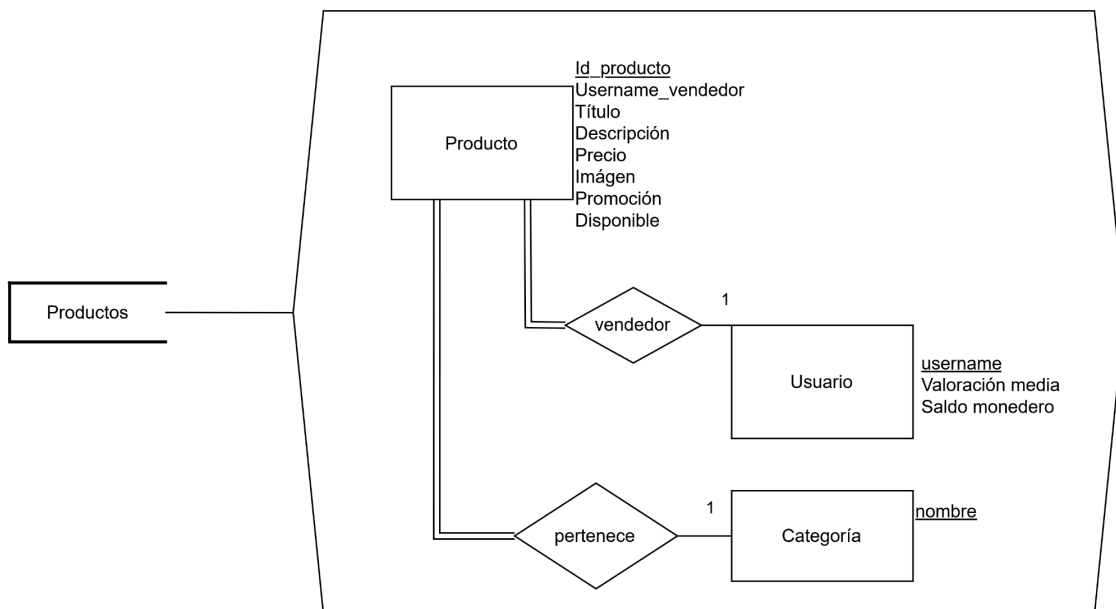
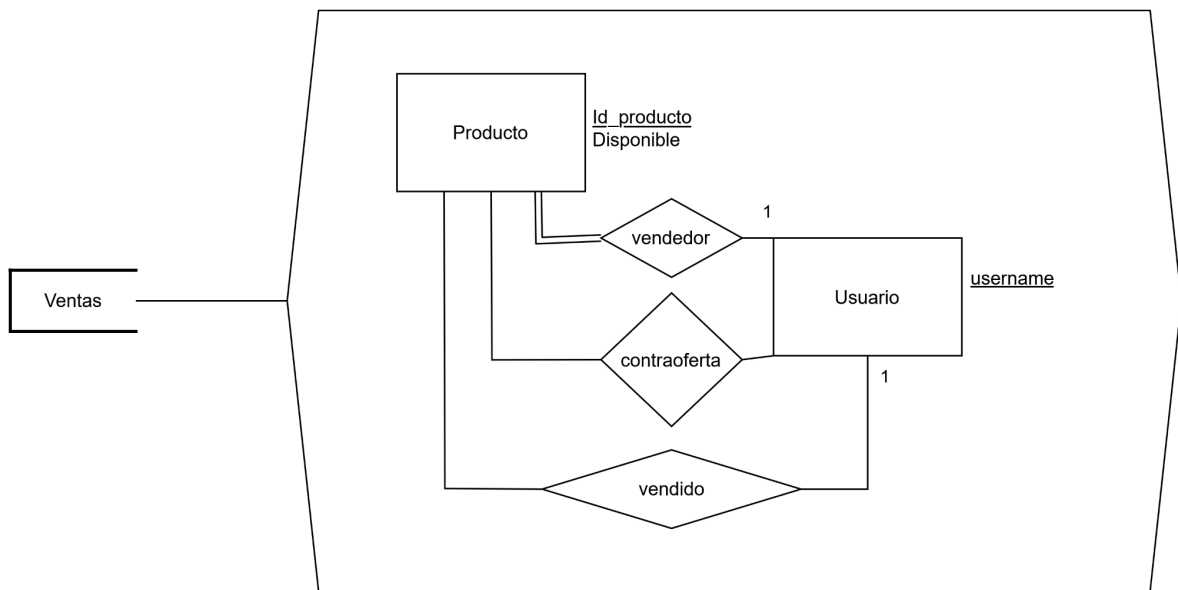




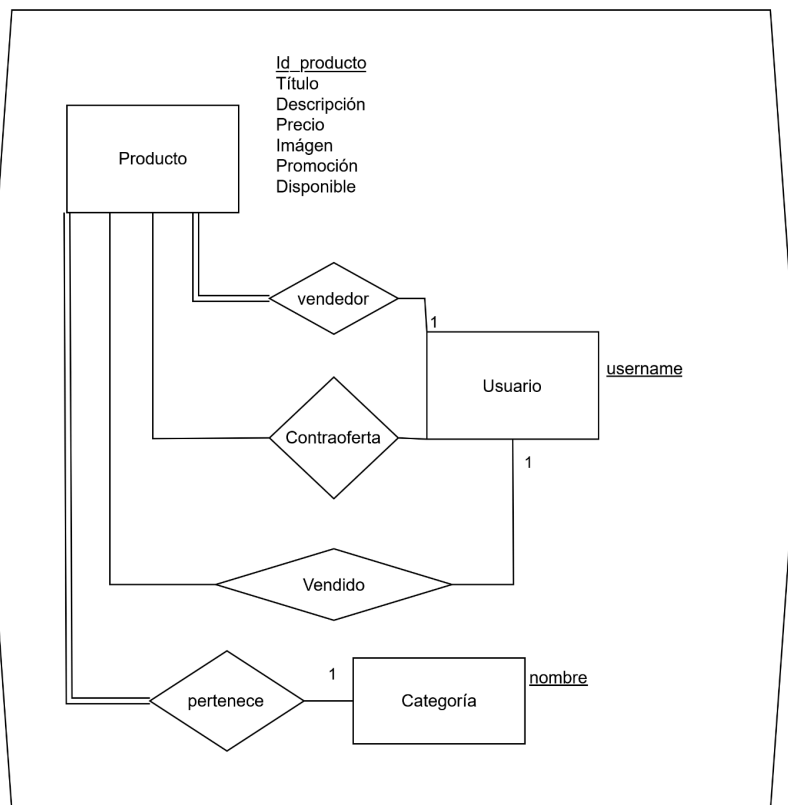
Esquema externo total del subsistema Gestión de perfiles:



4.4.2. Gestión de productos (Roberto González)

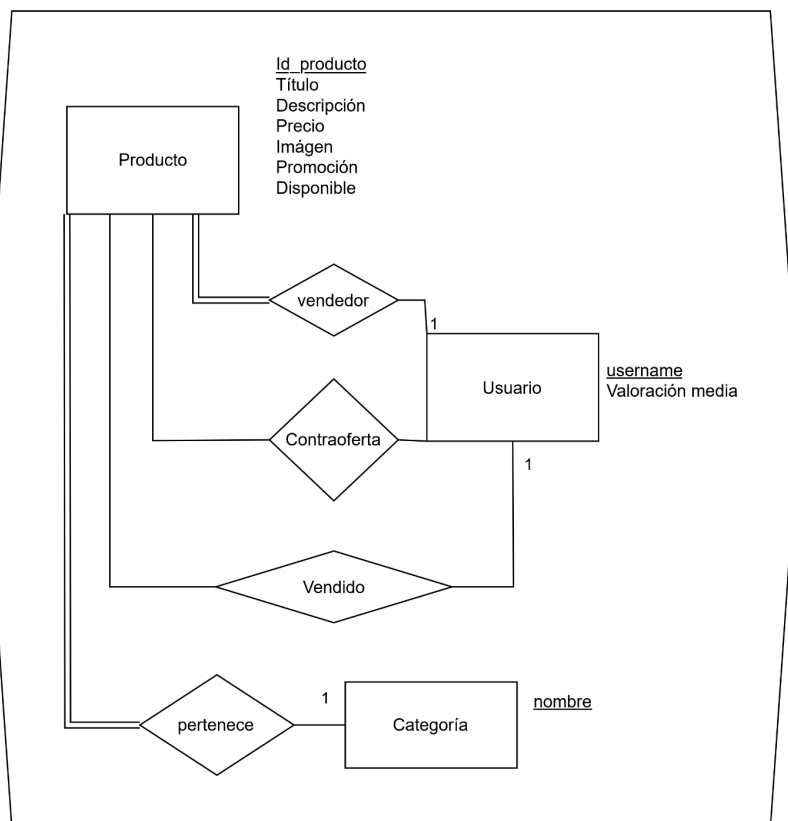


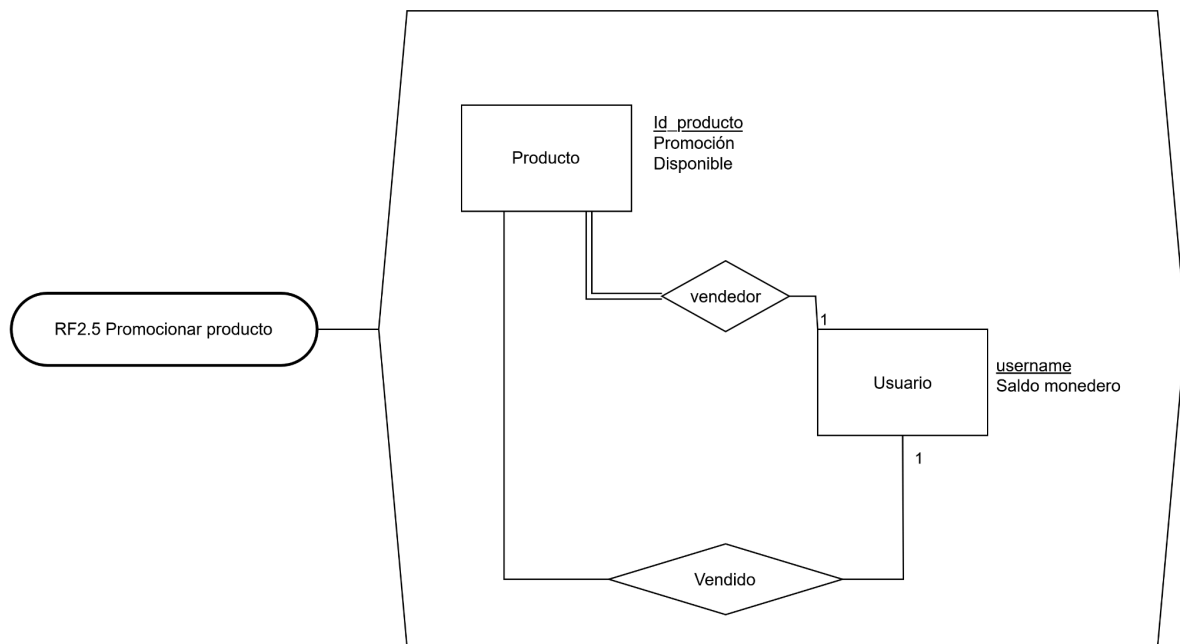
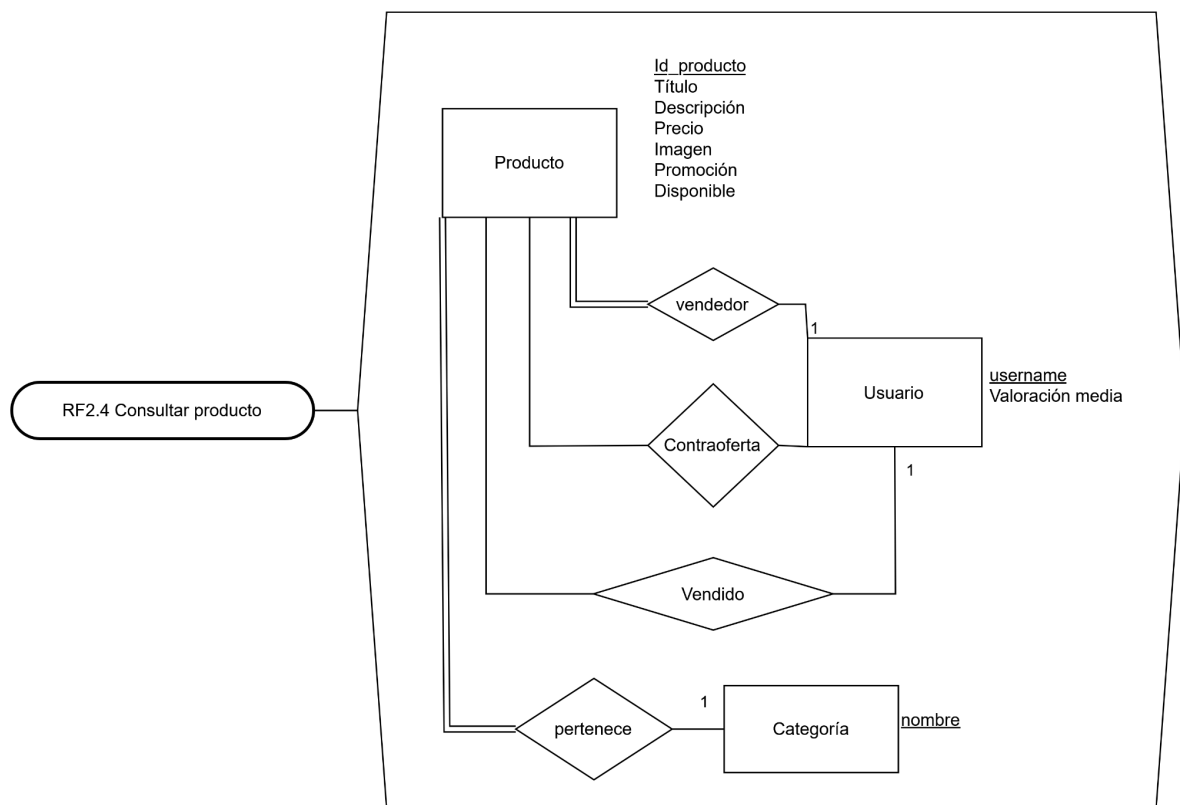
RF2.1 Dar de alta producto



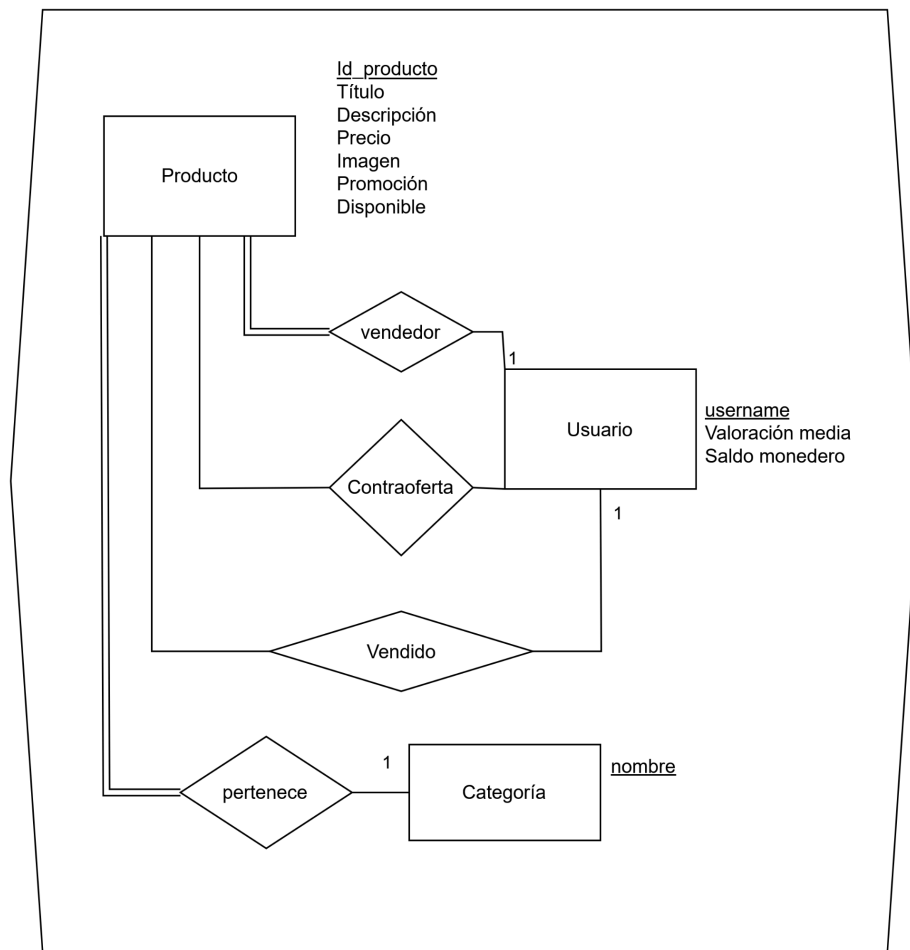
RF2.2 Modificar producto

RF2.3 Eliminar producto

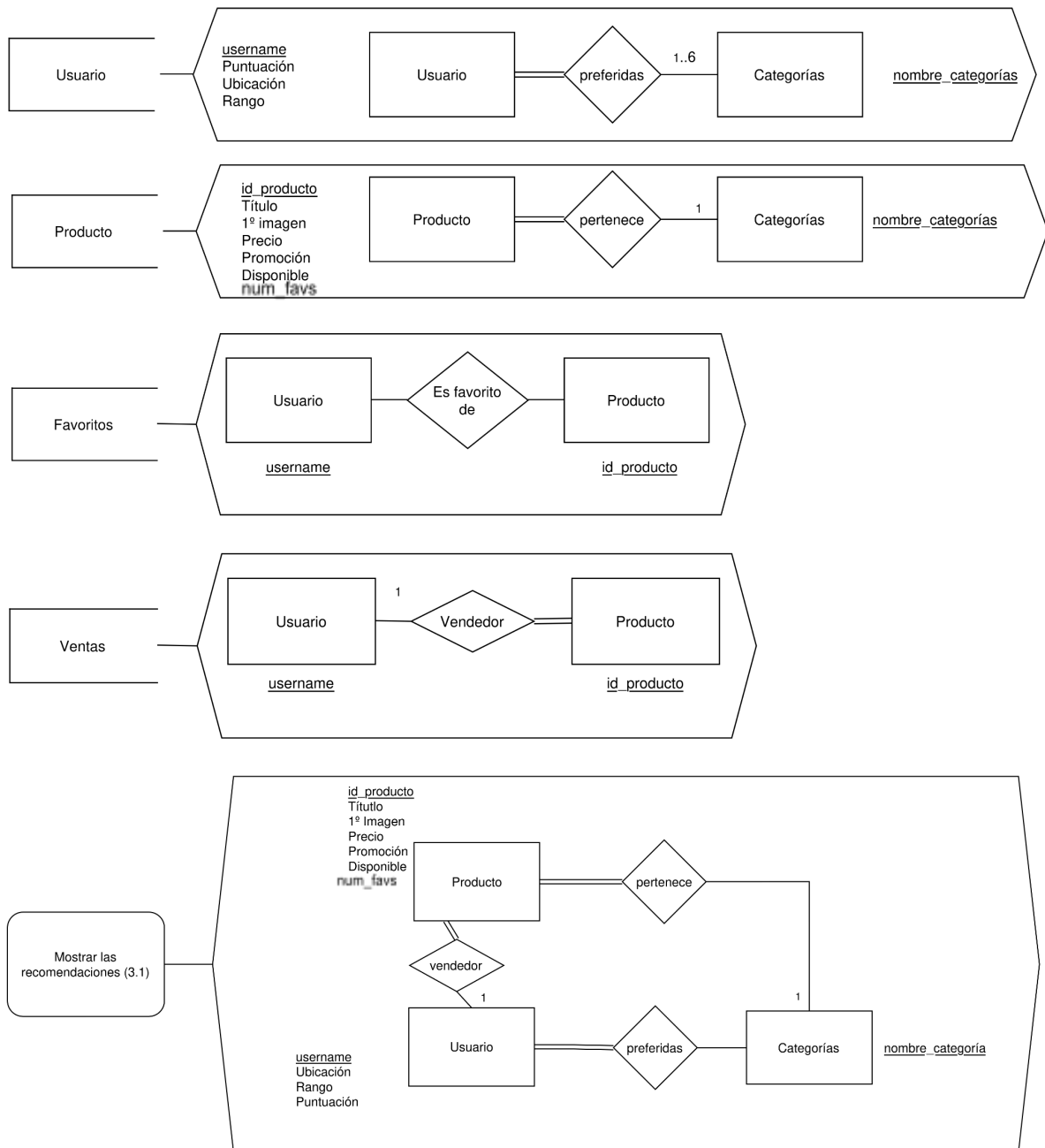


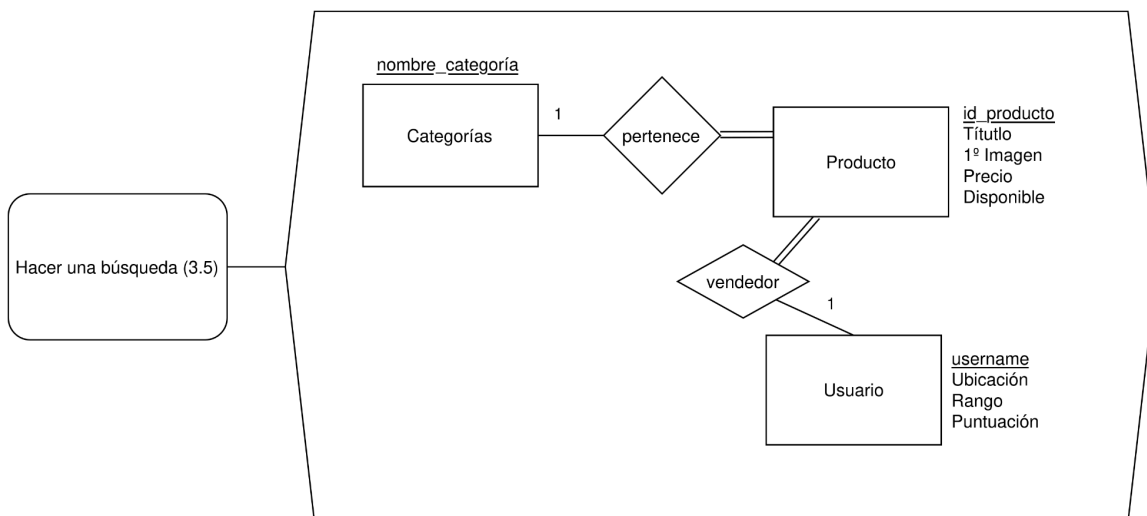
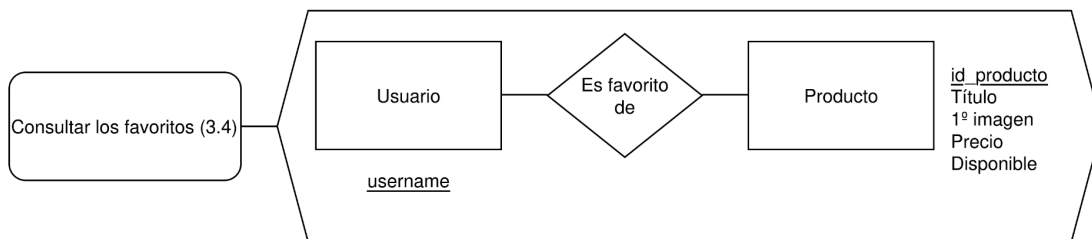
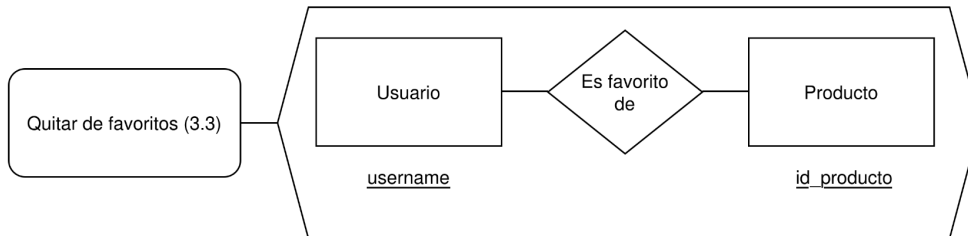
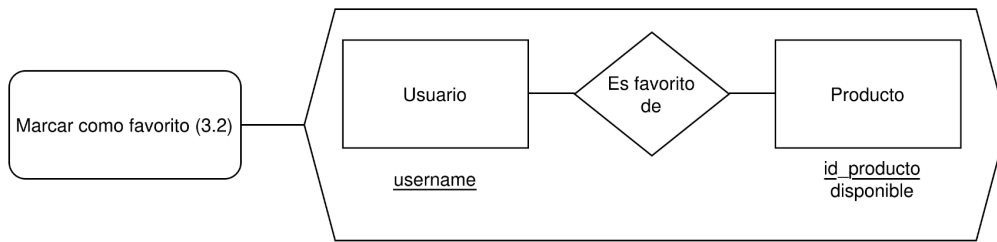


Esquema externo total del subsistema Gestión de productos:

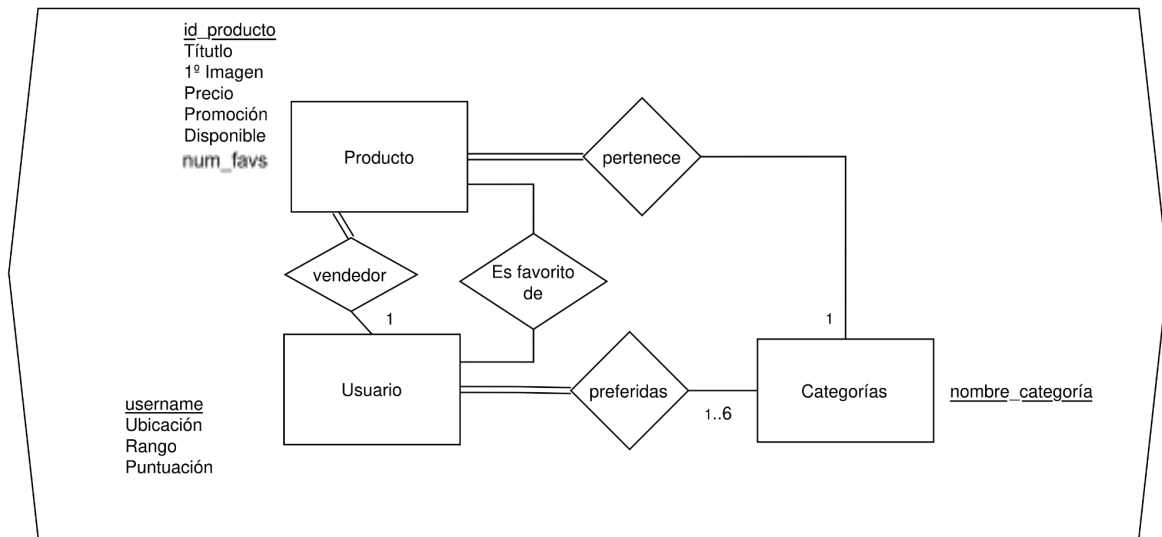


4.4.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo)

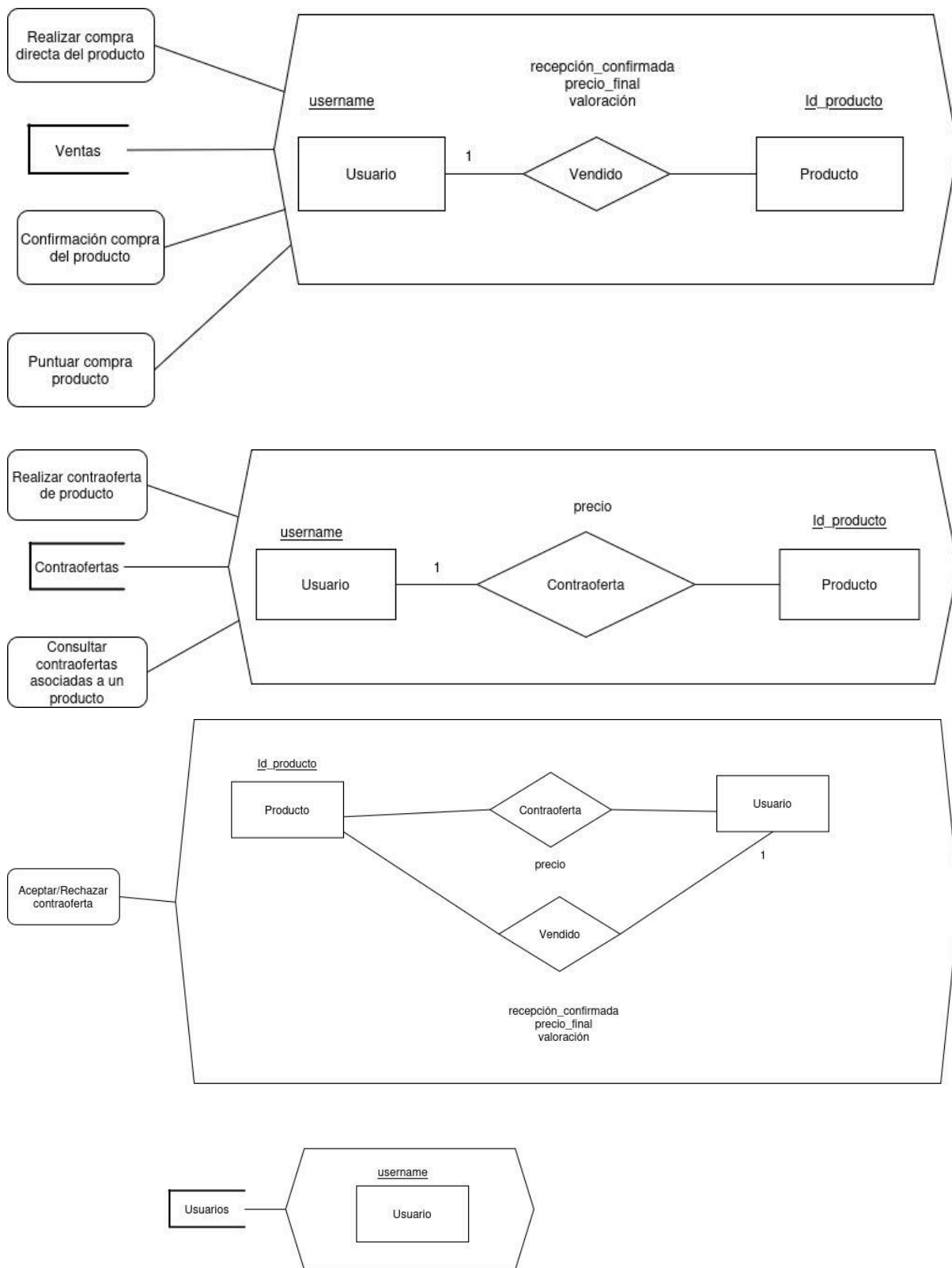




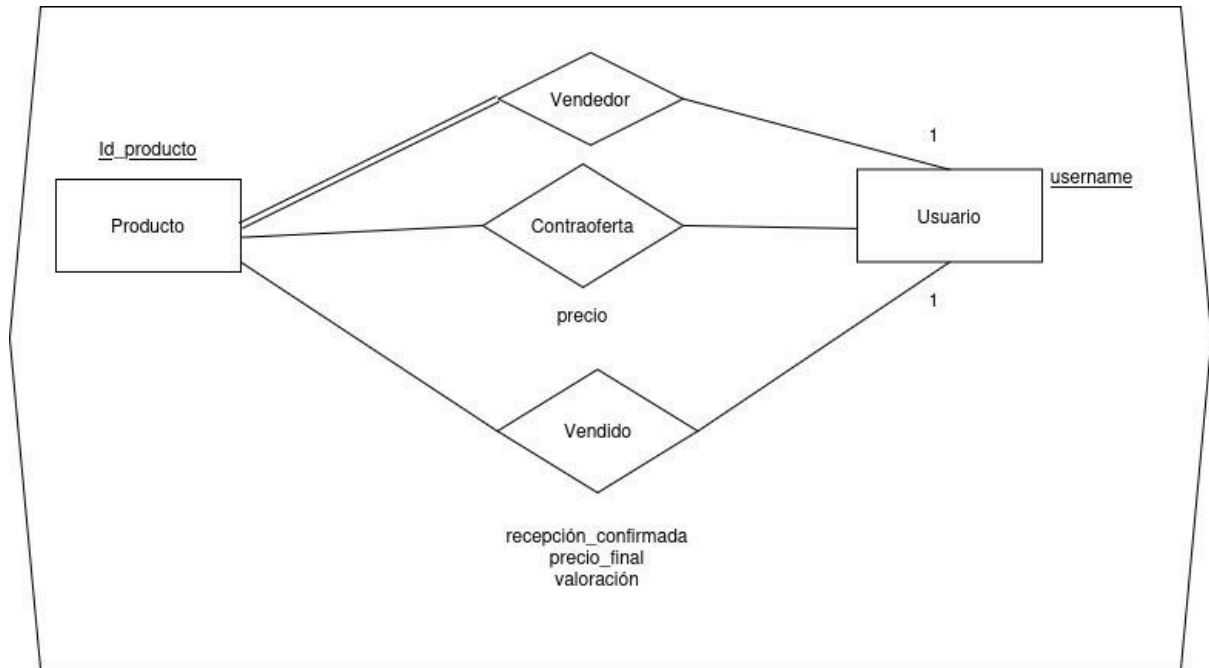
Esquema externo total del subsistema “Feed de recomendaciones, búsquedas y favoritos”:



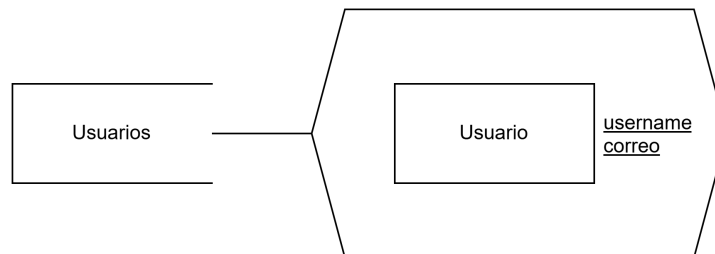
4.4.4. Gestión de ventas (Juan Manuel Fernández)

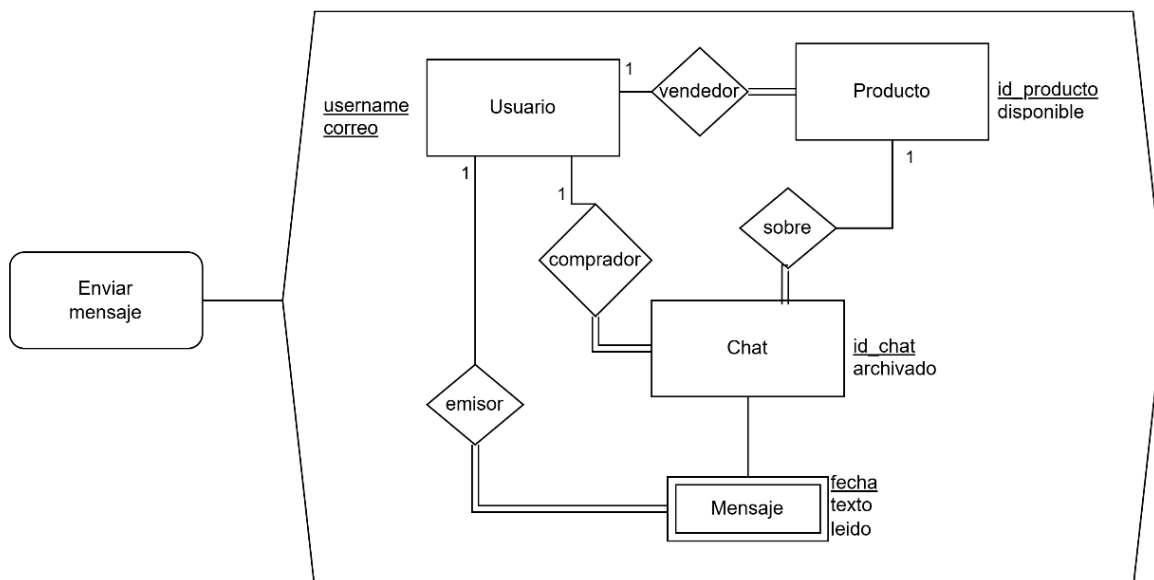
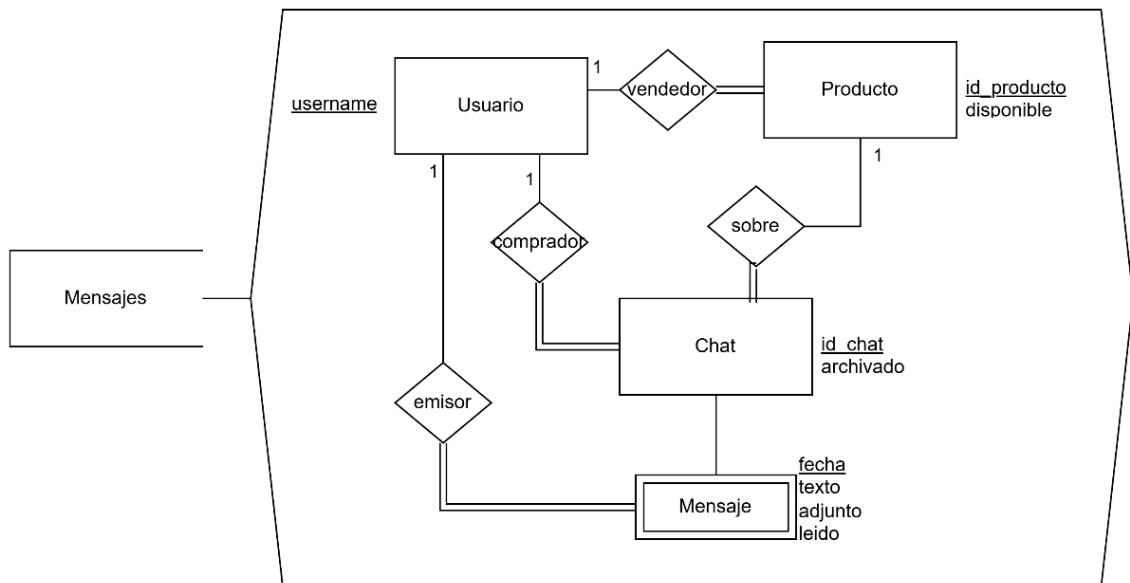
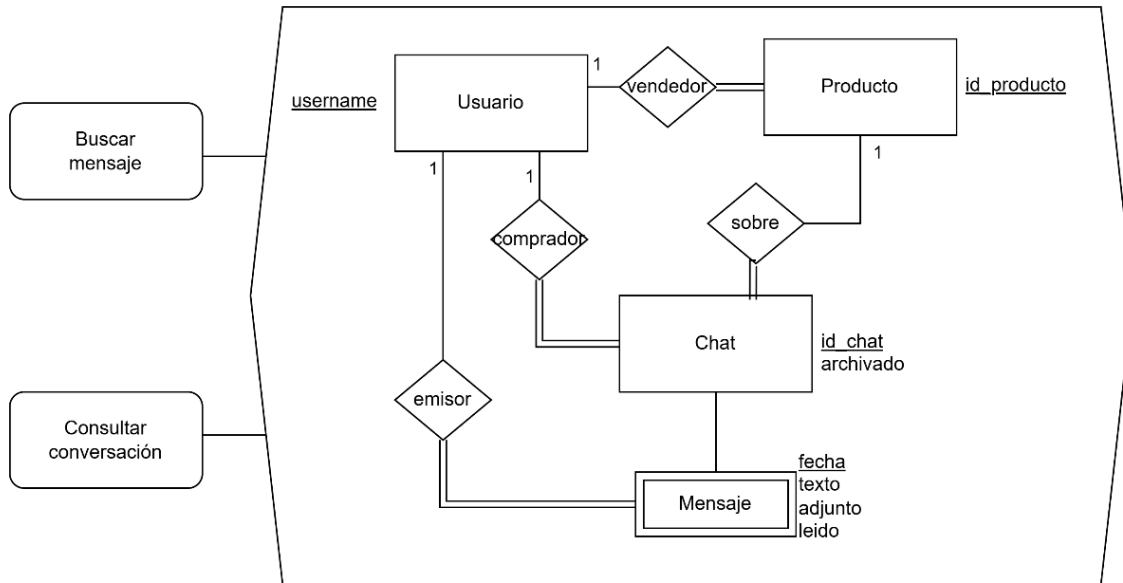


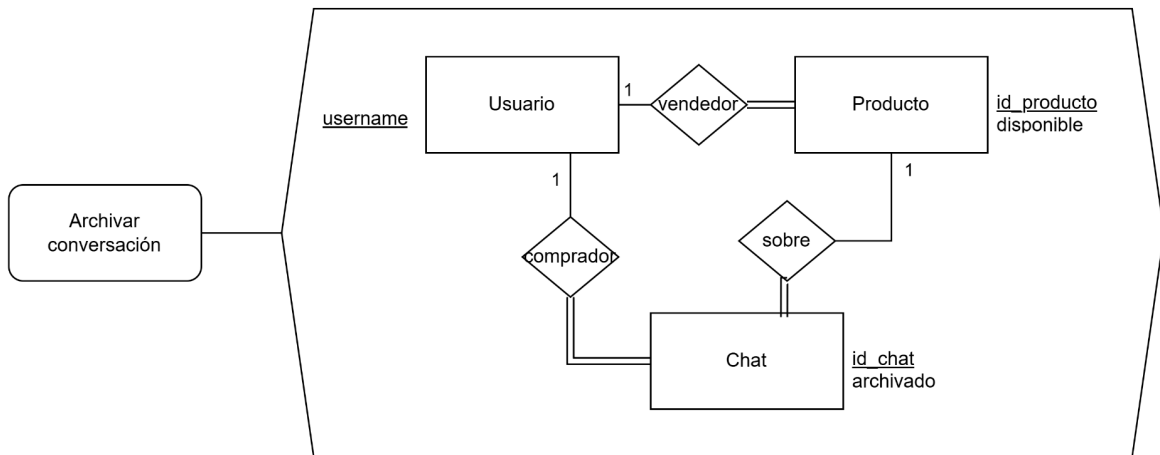
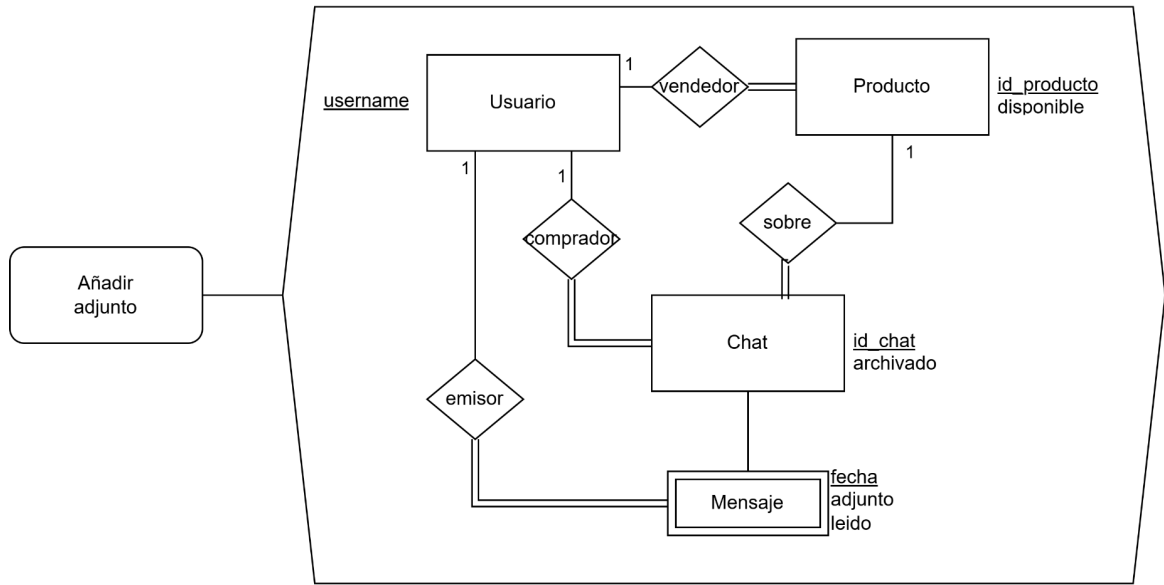
Esquema externo total del subsistema Ventas:



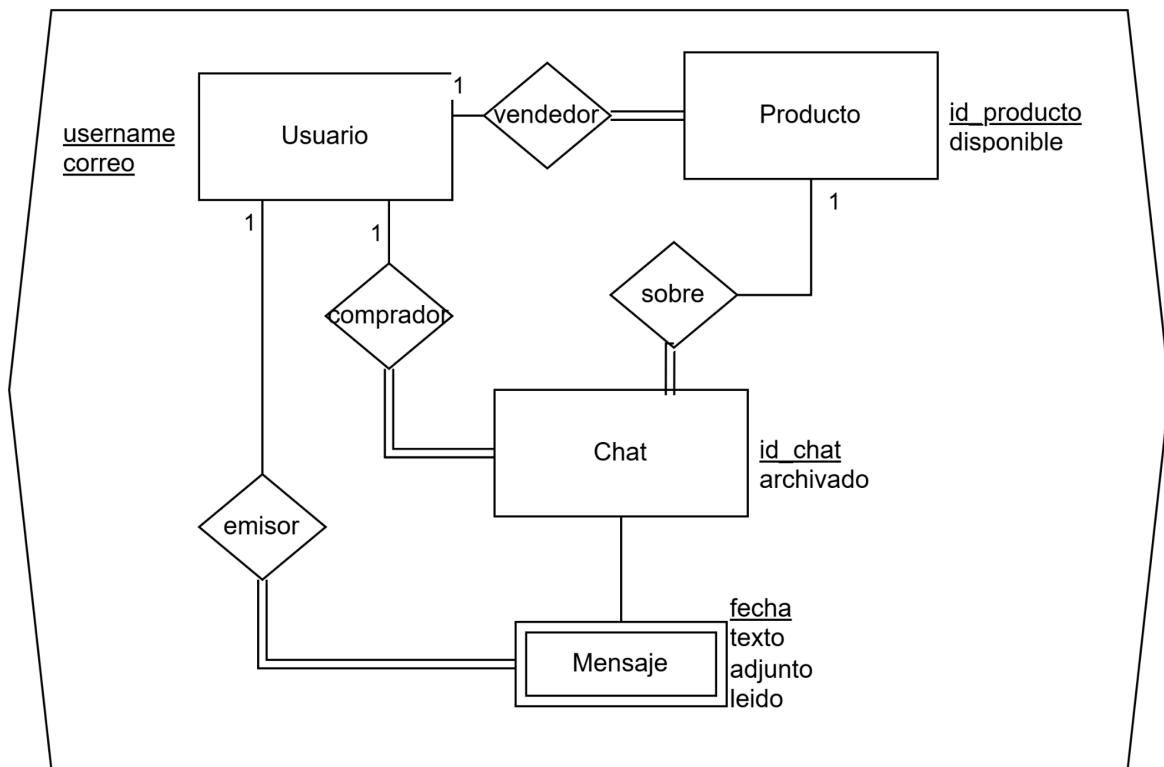
4.4.5. Gestión de mensajes (Aitor de la Iglesia)



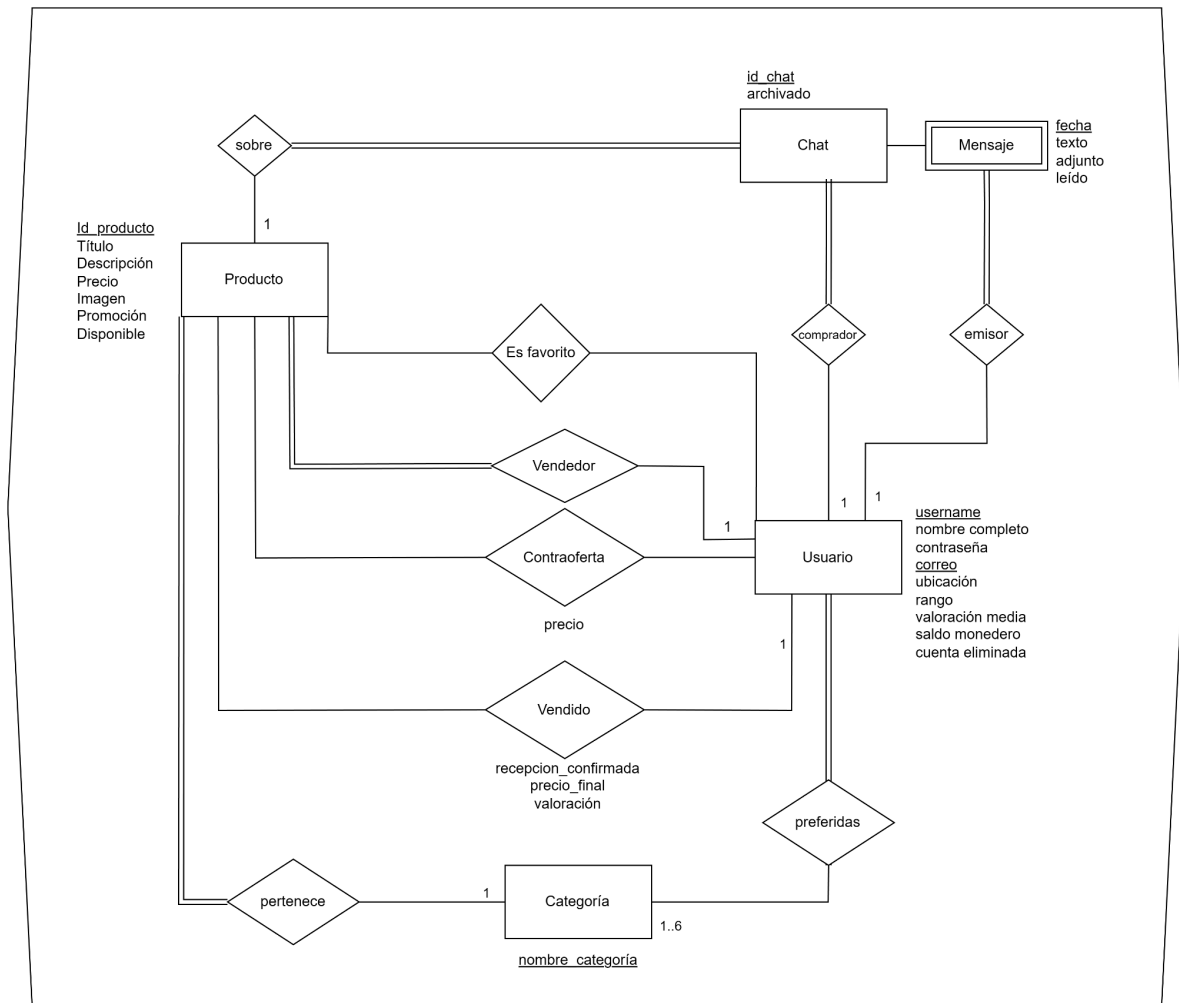




Esquema externo del subsistema de mensajes:



4.4.6. Sistema



4.5. Paso a tablas

A partir del esquema externo total hemos obtenido el siguiente conjunto de tablas. Las claves primarias irán subrayadas, las claves candidatas se señalan con "(CC)" y las claves externas con "(CE)".

- Usuario (username, correo (CC), nombre_completo, contraseña, ubicación, rango, valoración_media, saldo, cuenta_eliminada)
- Producto (id_producto, título, descripción, precio, imagen, promoción, disponible, num_favs)
- Categoría (nombre_cat)
- Preferidas (username (CE), nombre_cat (CE))
- Pertenece (id_producto (CE), nombre_cat (CE))

- Favorito (id_producto (CE), username (CE))
- Vendedor (id_producto (CE), username (CE))
- Contraoferta (id_producto (CE), username (CE), precio)
- Vendido (id_producto (CE), username (CE), recepción_confirmada, precio_final, valoración)
- Chat (id_chat, archivado)
- Mensaje (id_chat (CE), fecha, texto, adjunto, leído)
- Sobre (id_chat (CE), id_producto (CE))
- Comprador (id_chat (CE), username (CE))
- Emisor (id_chat (CE), fecha (CE compuesta), username (CE))

4.6. Fusión de tablas

Antes de estudiar las dependencias funcionales, fusionamos aquellas tablas de relaciones con cardinalidad 1:1 o 1:N, para simplificar el conjunto de tablas. De este modo, consideramos las siguientes fusiones:

- La relación *Emisor* entre *Usuario* y *Mensaje* es 1:N, pues cada mensaje es emitido por un único usuario y cada usuario emite muchos mensajes. Por tanto, fusionamos las tablas Emisor+Mensaje obteniendo:

Emisor+Mensaje (id_chat (CE), fecha, username (CE), texto, adjunto, leído)

- Las relaciones *Sobre* y *Comprador* entre *Chat* con *Producto* y *Usuario* respectivamente son ambas 1:N. Tenemos así una única entidad que sería un chat entre un comprador y un vendedor acerca de un producto. Por tanto, fusionamos las tablas Chat+Sobre+Comprador obteniendo:

Chat+Sobre+Comprador (id_chat, id_producto (CE), username (CE), archivado)

Donde (id_producto (CE), username (CE)) es una clave candidata compuesta.

- Las relaciones *Vendedor* y *Pertenece* entre *Producto* con *Usuario* y *Categoría* respectivamente son ambas 1:N. Tenemos así una única entidad que sería un producto que tiene un único vendedor y que pertenece a una única categoría. Por tanto, fusionamos las tablas Chat+Sobre+Comprador obteniendo:

Producto+Vendedor+Pertenece (id_producto, username (CE), nombre_cat (CE), título, descripción, precio, imagen, promoción, disponible, num_favs)

4.7. Dependencias funcionales

Analizando las tablas obtenidas, vemos que se tienen las siguientes dependencias funcionales:

- Usuario (username, correo (CC), nombre_completo, contraseña, ubicación, rango, valoración_media, saldo, cuenta_eliminada)
 - username → Usuario
 - correo → Usuario
- Producto+Vendedor+Pertenece (id_producto, username (CE), nombre_cat (CE), título, descripción, precio, imagen, promoción, disponible, num_favs)
 - id_producto → Producto+Vendedor+Pertenece
- Categoría (nombre_cat)
 - nombre_cat → Categoría
- Preferidas (username (CE), nombre_cat (CE))
 - (username, nombre_cat) → Preferidas
- Favorito (id_producto (CE), username (CE))
 - (id_producto, username) → Favorito
- Contraoferta (id_producto (CE), username (CE), precio)
 - (id_producto, username) → Contraoferta
- Vendido (id_producto (CE), username (CE), recepción_confirmada, precio_final, valoración)
 - (id_producto, username) → Vendido
- Chat+Sobre+Comprador (id_chat, id_producto (CE), username (CE), archivado)
 - id_chat → Chat+Sobre+Comprador
 - (id_producto, username) → Chat+Sobre+Comprador
- Emisor+Mensaje (id_chat (CE), fecha, username (CE), texto, adjunto, leído)
 - (id_chat, fecha) → Emisor+Mensaje

4.8. Normalización

Para cada tabla analizamos la clave primaria, las dependencias funcionales relevantes y si cumple las formas normales 1FN, 2FN, 3FN y BCNF.

Usuario (username, correo (CC), nombre_completo, contraseña, ubicación, rango, valoración_media, saldo, cuenta_eliminada)

(1FN): La tabla Usuario está en 1FN porque todos sus atributos son atómicos e indivisibles.

(2FN): El conjunto de atributos primos es {username, correo}, ya que estos dos atributos forman las únicas claves candidatas. Ambas claves son simples (no compuestas), por lo que no puede haber dependencias parciales. Todos los atributos no primos dependen completamente de username y también de correo. Por tanto, la tabla está en 2FN.

(3FN): Las dependencias no triviales son: {username $\rightarrow$ resto de atributos} y {correo $\rightarrow$ resto de atributos}. En ambas, el determinante es una clave candidata (superclave). Por tanto, se cumple que para toda dependencia $\alpha \rightarrow \beta$, α es una superclave. La tabla está en 3FN.

(FNBC): Dado que todos los determinantes de las dependencias funcionales son claves candidatas, se cumple la condición de Boyce-Codd. Por ello, la tabla Usuario está en FNBC.

Producto+Vendedor+Pertenece (id_producto, username (CE), nombre_cat (CE), título, descripción, precio, imagen, promoción, disponible, num_favs)

(1FN): La tabla está en 1FN porque todos sus atributos son atómicos.

(2FN): El atributo primo es únicamente id_producto, ya que es la única clave candidata y es simple. Todos los demás atributos dependen completamente de id_producto. Por tanto, la tabla está en 2FN.

(3FN): La única dependencia no trivial es {id_producto $\rightarrow$ resto de atributos}. El determinante id_producto es una superclave. Por tanto, la tabla está en 3FN.

(FNBC): La tabla cumple BCNF porque el único determinante es clave candidata. Producto+Vendedor+Pertenece está en FNBC.

Categoría (nombre_cat)

(1FN): Sólo tiene un atributo nombre_cat que es indivisible, por lo que la tabla está en 1FN.

(2FN): El único atributo primo es nombre_cat, clave primaria no compuesta. No hay más atributos. La tabla está en 2FN.

(3FN): La única dependencia no trivial tiene como determinante a una superclave. La tabla está en 3FN.

(FNBC): El determinante es clave candidata, por lo que también cumple FNBC.

Preferidas (username (CE), nombre_cat (CE))

(1FN): Los atributos son atómicos.

(2FN): La clave primaria es compuesta y ambos atributos son primos. No hay atributos no primos, por lo que no puede haber dependencias parciales. La tabla está en 2FN.

(3FN): Solo existe la dependencia $\{(username, nombre_cat) \rightarrow \text{resto de atributos}\}$. El determinante es clave primaria, luego es superclave. La tabla está en 3FN.

(FNBC): Como el determinante de la única dependencia es una superclave, la tabla está en FNBC.

Favorito (id_producto (CE), username (CE))

(1FN): Los atributos son indivisibles.

(2FN): La clave es compuesta (id_producto, username). No existen atributos no primos. La tabla está en 2FN.

(3FN): La única dependencia tiene como determinante la clave primaria. La tabla está en 3FN.

(FNBC): El determinante es superclave. La tabla está en FNBC.

Contraoferta (id_producto (CE), username (CE), precio)

(1FN): Todos los atributos son atómicos.

(2FN): La clave primaria es compuesta: (id_producto, username). Los atributos no primos (precio, fecha) dependen completamente de la clave compuesta. Por ello, la tabla está en 2FN.

(3FN): La única dependencia no trivial tiene como determinante a una superclave. La tabla está en 3FN.

(FNBC): El determinante es clave candidata. Por tanto, Contraoferta está en FNBC.

Vendido (id_producto (CE), username (CE), recepción_confirmada, precio_final, valoración)

(1FN): La tabla está en 1FN.

(2FN): Clave primaria compuesta. Los atributos no primos dependen de toda la clave. La tabla está en 2FN.

(3FN): La única dependencia no trivial tiene como determinante a una superclave. La tabla está en 3FN.

(FNBC): El determinante es clave candidata. La tabla está en FNBC.

Chat+Sobre+Comprador (id_chat, id_producto (CE), username (CE), archivado)

(1FN): Todos los atributos son indivisibles.

(2FN): Atributos primos: {id_chat} y {id_producto, username}. Todos los atributos no primos (archivado) dependen completamente de ambas claves candidatas. La tabla está en 2FN.

(3FN): Analizamos las dependencias:

1. id_chat → resto: id_chat es superclave
2. (id_producto, username) → id_chat: (id_producto, username) es clave candidata luego superclave

En ambos casos el determinante es superclave. La tabla está en 3FN.

(FNBC): Todas las dependencias tienen como determinante una clave candidata. Por tanto, la tabla está en FNBC.

Emisor+Mensaje (id_chat (CE), fecha, username (CE), texto, adjunto, leído)

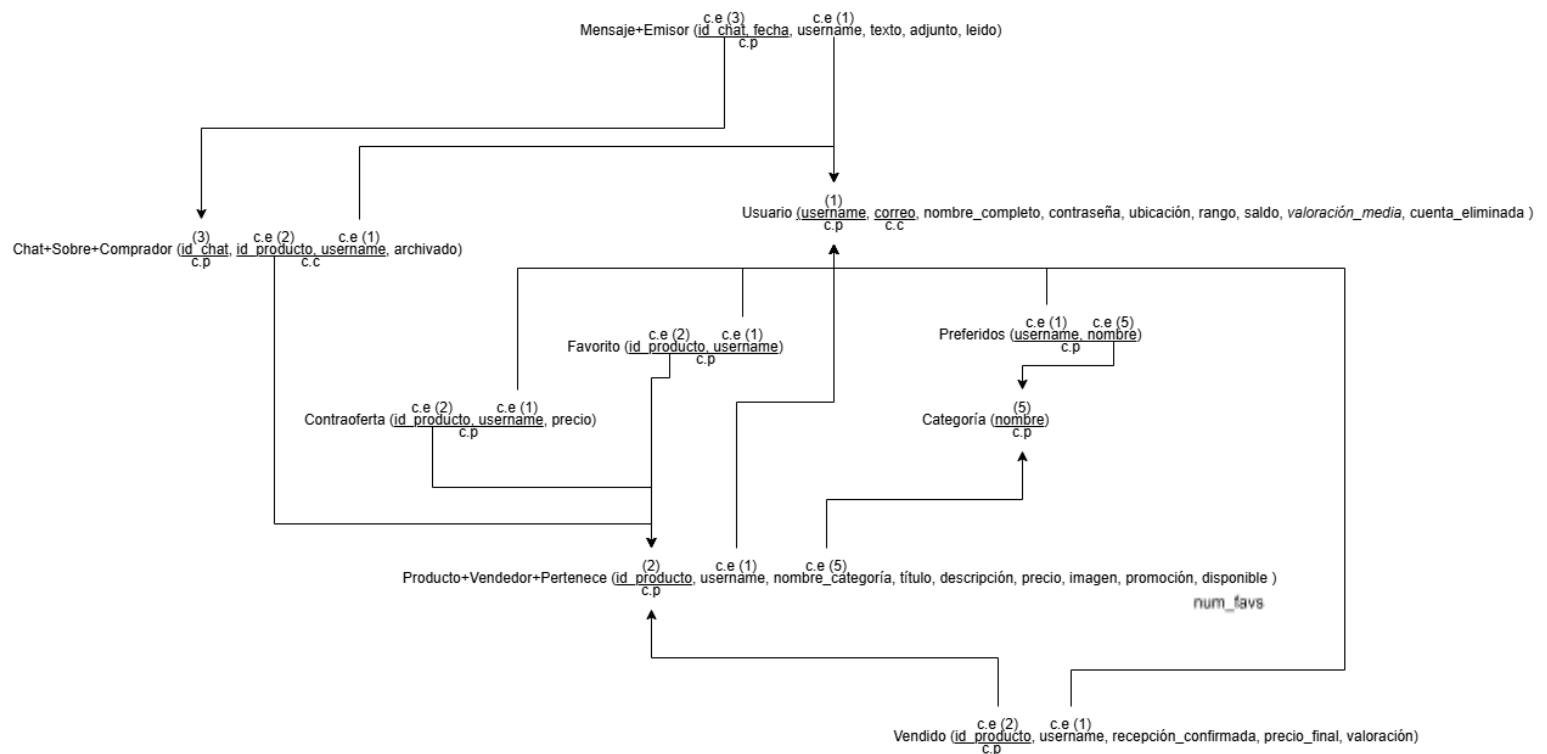
(1FN): Todos los atributos son atómicos.

(2FN): Clave primaria compuesta: (id_chat, fecha). No existen dependencias parciales, pues no hay dependencias de un solo atributo de la clave. La tabla está en 2FN.

(3FN): La única dependencia no trivial tiene como determinante a una superclave. Por ello, la tabla está en 3FN.

(FNBC): El determinante es clave primaria luego está en FNBC.

El esquema resultante de nuestro análisis es:



Vemos que el modelo relacional está en FNBC, ya que todas las dependencias que tenemos tienen a la izquierda una superclave.

5. Creación de tablas

Las sentencias para la creación de tablas que hemos aplicado en nuestra BD son:

```
CREATE TABLE Usuario(
  username VARCHAR2(128) PRIMARY KEY,
  correo VARCHAR2(128) UNIQUE,
  nombre_completo VARCHAR2(128),
  contraseña VARCHAR2(128),
  ubi_latitud FLOAT(6),
  ubi_longitud FLOAT(6),
  rango FLOAT(6),
  saldo FLOAT(10),
  valoracion_media FLOAT(3),
  cuenta_eliminada INT
);
```

```
CREATE TABLE Categoria(
  nombre VARCHAR2(128) PRIMARY KEY
);
```

```

CREATE TABLE Producto(
    id_producto INT PRIMARY KEY,
    username VARCHAR2(128),
    nombre_categoria VARCHAR2(128),
    titulo VARCHAR2(128) NOT NULL,
    descripcion VARCHAR2(512),
    precio FLOAT(10) NOT NULL,
    imagen BLOB,
    promocion FLOAT(2),
    num_favs INT,
    disponible INT,
    CONSTRAINT FK_ProductoVendedor FOREIGN KEY (username) REFERENCES
Usuario(username),
    CONSTRAINT FK_ProductoCategoria FOREIGN KEY (nombre_categoria) REFERENCES
Categoria(nombre)
);

CREATE TABLE Chat(
    id_chat INT PRIMARY KEY,
    id_producto INT,
    username VARCHAR2(128),
    archivado INT,
    CONSTRAINT PK_UsernameIDProd UNIQUE (id_producto, username)
);

CREATE TABLE Mensaje(
    id_chat INT NOT NULL,
    fecha TIMESTAMP(6) NOT NULL,
    username VARCHAR2(128) NOT NULL,
    texto VARCHAR2(512),
    adjunto BLOB,
    leído INT,
    CONSTRAINT PK_Mensaje PRIMARY KEY (id_chat, fecha),
    CONSTRAINT FK_MensajeEmisor FOREIGN KEY (username) REFERENCES
Usuario(username),
    CONSTRAINT FK_MensajeID_chat FOREIGN KEY (id_chat) REFERENCES
Chat(id_chat),
    CONSTRAINT NotNull_Texto_Or_Adjunto CHECK (texto IS NOT NULL OR adjunto
IS NOT NULL)
);

CREATE TABLE Favorito(
    id_producto INT,
    username VARCHAR2(128),
    CONSTRAINT PK_Favorito PRIMARY KEY (id_producto, username),
    CONSTRAINT FK_FavoritoProducto FOREIGN KEY (id_producto) REFERENCES
Producto(id_producto),

```

```

    CONSTRAINT FK_FavoritoUsuario FOREIGN KEY (username) REFERENCES
Usuario(username)
);

CREATE TABLE Contraoferta(
    id_producto INT,
    username VARCHAR2(128),
    precio FLOAT(10) NOT NULL,
    CONSTRAINT PK_Contraoferta PRIMARY KEY (id_producto,username),
    CONSTRAINT FK_ContraofertaProducto FOREIGN KEY (id_producto) REFERENCES
Producto(id_producto),
    CONSTRAINT FK_ContraofertaContraofertante FOREIGN KEY (username)
REFERENCES Usuario(username)
);

CREATE TABLE Preferidos(
    username VARCHAR(128),
    nombre VARCHAR2(128),
    CONSTRAINT PK_Preferidos PRIMARY KEY (username,nombre),
    CONSTRAINT FK_PreferidosUsuario FOREIGN KEY (username) REFERENCES
Usuario(username),
    CONSTRAINT FK_PreferidosCategoria FOREIGN KEY (nombre) REFERENCES
Categoria(nombre)
);

CREATE TABLE Vendido(
    id_producto INT PRIMARY KEY,
    username VARCHAR2(128),
    recepcion_confirmada INT,
    precio_final FLOAT(10),
    valoracion INT,
    CONSTRAINT FK_VendidoProducto FOREIGN KEY (id_producto) REFERENCES
Producto(id_producto),
    CONSTRAINT FK_VendidoComprador FOREIGN KEY (username) REFERENCES
Usuario(username)
);

-- Categorías maestras del sistema
INSERT INTO Categoria VALUES ('Vehículos');
INSERT INTO Categoria VALUES ('Moda');
INSERT INTO Categoria VALUES ('Tecnología');
INSERT INTO Categoria VALUES ('Deportes');
INSERT INTO Categoria VALUES ('Hogar');
INSERT INTO Categoria VALUES ('Libros');

```

6. Descripción de las transacciones

6.1. Gestión de Perfiles (Elsa Rodríguez)

- **Registro de Usuario**

Esta transacción involucra múltiples operaciones que deben ejecutarse de forma atómica. Primero se verifica que no exista otro usuario con el mismo username o correo electrónico, a continuación se inserta el nuevo registro en la tabla Usuario con todos sus datos personales, y finalmente se insertan las categorías preferidas del usuario en la tabla Preferidos. Si cualquiera de estas operaciones falla, por ejemplo porque el username ya existe o porque una categoría no es válida, debe deshacerse todo el trabajo realizado. El código implementa esta atomicidad mediante el uso de savepoints y rollback, asegurando que no queden datos inconsistentes en caso de error.

- **Inicio de Sesión**

Esta transacción se ejecuta cada vez que un usuario intenta acceder a la plataforma. Consiste en una consulta a la base de datos que recupera la contraseña almacenada para el username proporcionado y la compara con la contraseña introducida por el usuario. Aunque es una operación de solo lectura, constituye una transacción que debe ejecutarse de forma aislada para garantizar una visión consistente de los datos, evitando que lecturas concurrentes de otras transacciones que pudieran estar modificando las credenciales del usuario afecten al resultado. La transacción devuelve un valor booleano indicando si las credenciales son válidas o no.

- **Modificación de Perfil**

Esta transacción permite modificar los datos personales del usuario como el nombre completo, correo electrónico, ubicación geográfica y rango de búsqueda. Aunque se trata de una única sentencia UPDATE, constituye una transacción que debe confirmarse mediante commit para que los cambios persistan en la base de datos. En caso de fallo, el sistema garantiza que los datos anteriores se mantengan intactos.

- **Actualización de categorías preferidas**

Esta transacción reemplaza completamente las categorías preferidas de un usuario. Primero elimina todas las categorías existentes del usuario en la tabla Preferidos y luego inserta las nuevas categorías seleccionadas. Es fundamental que ambas operaciones se ejecuten como una unidad indivisible, puesto que si se eliminaran las categorías antiguas pero fallara la inserción de las nuevas, el usuario quedaría sin categorías preferidas, violando la regla de negocio que exige al menos una categoría.

- **Eliminación lógica de cuenta**

La baja de un usuario es un proceso que requiere múltiples validaciones antes de poder ejecutarse. Cuando un usuario solicita darse de baja, el sistema primero verifica que no tenga productos en venta activos consultando la tabla Producto para buscar registros donde el username coincida y el campo disponible sea igual a uno. Luego verifica que el usuario no tenga ventas pendientes como comprador, consultando la tabla Vendido para buscar registros donde el username coincida y el campo recepcion_confirmada sea igual a cero. También se comprueba que el usuario no tenga contraofertas activas consultando la tabla Contraoferta. Si alguna de estas condiciones no se cumple, el sistema lanza un error indicando el motivo por el cual no puede darse de baja. Si todas las validaciones son exitosas, se procede a desactivar todos los productos del usuario actualizando el campo disponible a cero en la tabla Producto. Posteriormente, se actualiza la tabla Usuario vaciando los campos personales sensibles, estableciendo el nombre como "Usuario Eliminado", invalidando la contraseña y marcando la cuenta como eliminada.. También se eliminan las categorías preferidas del usuario de la tabla Preferidos. Finalmente, se realiza un commit para confirmar todos los cambios.

- **Actualización de saldo del monedero**

El monedero virtual permite a los usuarios gestionar su saldo dentro de la plataforma. Cuando un usuario desea recargar o retirar saldo, proporciona la cantidad a añadir o retirar, y la contraseña en el segundo caso. El sistema valida que la cantidad sea un número positivo mayor que cero. Si la validación es exitosa, se actualiza el campo saldo en la tabla Usuario sumando o restando la cantidad proporcionada al saldo actual. Se realiza un commit para confirmar el cambio. De manera similar, cuando se realiza una compra, el sistema verifica que el comprador tenga saldo suficiente consultando el campo saldo de la tabla Usuario. Si el saldo es suficiente, se descuenta el precio del producto del saldo del comprador y se añade al saldo del vendedor mediante dos operaciones de actualización en la tabla Usuario.

- **Consultas de datos (usuario, categorías preferidas, categorías disponibles)**

Estas operaciones de lectura también son transacciones que, aunque no modifican datos, deben garantizar una visión consistente de la base de datos. El aislamiento transaccional asegura que las lecturas no se vean afectadas por otras transacciones concurrentes que pudieran estar modificando los mismos datos.

6.2. Gestión de Productos (Roberto González)

- **Publicación de Producto**

Cuando un vendedor desea publicar un nuevo producto, proporciona los datos del producto a través del formulario correspondiente. El sistema valida que el título no esté vacío y no supere los 80 caracteres, que la descripción no supere los 500 caracteres, que el precio sea un número positivo, y que la categoría seleccionada exista en la tabla Categoría. Si todas las validaciones son exitosas, se genera un nuevo identificador de producto consultando el

máximo identificador actual en la tabla Producto y sumándole uno. A continuación, se inserta un nuevo registro en la tabla Producto con el identificador generado, el nombre de usuario del vendedor, la categoría, el título, la descripción, el precio, la imagen si se proporciona, la promoción inicializada a cero, el número de favoritos inicializado a cero, y el campo disponible establecido a uno. Finalmente, se realiza un commit para confirmar la inserción.

- **Modificación de Producto**

El proceso de modificación de un producto permite al vendedor actualizar la información de sus productos publicados. Cuando el vendedor solicita modificar un producto, el sistema primero verifica que el producto exista, esté disponible, no tenga contraofertas activas y pertenezca al usuario que realiza la solicitud comparando el campo username. A continuación, valida los nuevos datos proporcionados de manera similar a la publicación. Si las validaciones son exitosas, se actualiza el registro correspondiente en la tabla Producto con los nuevos valores de título, descripción, precio, categoría e imagen. Se realiza un commit para confirmar los cambios.

- **Eliminación de Producto**

Cuando un vendedor desea eliminar un producto, el sistema primero verifica que el producto exista, esté disponible, no tenga contraofertas activas y pertenezca al usuario. Se realiza una eliminación lógica estableciendo el campo disponible a cero en la tabla Producto. El producto deja de aparecer en búsquedas y listados pero se mantiene el registro por simplicidad. Se realiza un commit para confirmar el cambio.

- **Gestión de Promociones**

Los vendedores pueden promocionar sus productos para aumentar su visibilidad en el feed de búsqueda. Cuando un vendedor desea promocionar un producto, proporciona el grado de promoción deseado (valor entre 0 y 1). El sistema valida que el nuevo grado sea mayor que la promoción actual del producto. El coste de la promoción se calcula como el incremento del grado multiplicado por 0.1 y por el precio del producto. Si el vendedor tiene saldo suficiente en su monedero, se descuenta el coste y se actualiza el campo promocion en la tabla Producto. Los productos con mayor grado de promoción aparecen con mayor prioridad en los resultados de búsqueda. Se realiza un commit para confirmar el cambio.

6.3. Feed de recomendaciones, búsquedas y favoritos (Daniel Hidalgo)

- **Gestión de Favoritos**

Cuando un usuario añade un producto a favoritos, el sistema primero verifica que el producto exista y esté disponible. Luego comprueba que el usuario no haya añadido ya ese producto a favoritos consultando la tabla Favorito. Si las verificaciones son exitosas, se inserta un nuevo registro en la tabla Favorito con el identificador del producto y el nombre de usuario. Además, con los disparadores que luego se mencionan, se incrementa el campo num_favs en la tabla Producto para reflejar el nuevo número de favoritos. Cuando un usuario elimina un producto de favoritos (o cuando elimina su cuenta), se elimina el registro correspondiente de la tabla Favorito y se decrementa el campo num_favs en la tabla Producto.

6.4. Gestión de Ventas (Juan Manuel Fernández)

- **Realizar Contraoferta**

Cuando un comprador desea realizar una contraoferta sobre un producto, proporciona el precio que está dispuesto a pagar. El sistema primero verifica que el producto exista y esté disponible consultando la tabla Producto. También verifica que el comprador no sea el vendedor del producto comparando los nombres de usuario. A continuación, comprueba si ya existe una contraoferta del mismo usuario para ese producto consultando la tabla Contraoferta. Si ya existe, se informa al usuario que ya hay una contraoferta existente. Si no existe, se inserta un nuevo registro en la tabla Contraoferta con el identificador del producto, el nombre de usuario del comprador y el precio ofrecido. Se realiza un commit para confirmar la operación.

- **Aceptar Contraoferta**

Cuando un vendedor acepta una contraoferta, el sistema inicia una transacción que involucra múltiples operaciones. Primero verifica que la contraoferta exista consultando la tabla Contraoferta y que el usuario que acepta sea el vendedor del producto. Luego verifica que el comprador tenga saldo suficiente para pagar el precio acordado consultando el campo saldo en la tabla Usuario. Si el saldo es insuficiente, se lanza un error. Si todas las verificaciones son exitosas, se descuenta el precio del saldo del comprador, se inserta un nuevo registro en la tabla Vendido con el identificador del producto, el nombre de usuario del comprador, el campo recepcion_confirmada establecido a cero, el precio final acordado y la valoración inicializada a 0. Se actualiza el campo disponible del producto a cero en la tabla Producto para marcarlo como no disponible. Se eliminan todas las contraofertas asociadas al producto de la tabla Contraoferta. Finalmente, se realiza un commit para confirmar todos los cambios.

- **Rechazar Contraoferta**

Cuando un vendedor rechaza una contraoferta, el sistema verifica que la contraoferta exista y que el usuario que rechaza sea el vendedor del producto. Si las verificaciones son exitosas, se elimina el registro correspondiente de la tabla Contraoferta. Se realiza un commit para confirmar la eliminación.

- **Compra Directa**

Cuando un comprador decide comprar un producto al precio publicado sin realizar contraoferta, el sistema inicia una transacción similar a la aceptación de contraoferta. Verifica que el producto exista y esté disponible, que el comprador no sea el vendedor, y que el comprador tenga saldo suficiente. Si todas las verificaciones son exitosas, se retira el dinero correspondiente del comprador, se crea el registro en la tabla Vendido con el precio original del producto teniendo en cuenta la promoción si la hubiera, se marca el producto como no disponible, y se eliminan las contraofertas existentes si las hubiera. Se realiza un commit para confirmar la transacción.

- **Confirmar Recepción**

Cuando un comprador confirma que ha recibido el producto comprado, el sistema verifica que exista un registro en la tabla Vendido asociado al comprador y al producto, y que el campo recepcion_confirmada sea igual a cero. Si la verificación es exitosa, se actualiza el campo recepcion_confirmada a uno en la tabla Vendido. Esta confirmación es importante porque hasta que no se confirme la recepción, tanto el comprador como el vendedor tienen restricciones para darse de baja del sistema. Se realiza un commit para confirmar el cambio.

- **Puntuar Venta**

Después de confirmar la recepción de un producto, el comprador puede valorar la transacción. El sistema verifica que la recepción haya sido confirmada consultando el campo recepcion_confirmada en la tabla Vendido. Si está confirmada, se actualiza el campo valoracion con el valor proporcionado por el comprador, que debe estar entre uno y cinco. Posteriormente, se recalcula la valoración media del vendedor consultando todas las valoraciones de sus productos vendidos en la tabla Vendido y calculando el promedio. Este promedio se actualiza en el campo valoracion_medio de la tabla Usuario correspondiente al vendedor. Se realiza un commit para confirmar los cambios.

6.5. Gestión de Mensajes (Aitor de la Iglesia)

- **Crear Chat**

Cuando un comprador inicia una conversación con un vendedor sobre un producto, el sistema primero verifica que se pueda crear, es decir, verifica si: ya existe un chat entre ambos usuarios para ese producto, si los dos usuarios son el mismo y si el producto por el que se habla está disponible. Si ya existe, se utiliza el chat existente. Si no existe, se genera un nuevo identificador de chat consultando el máximo identificador actual y sumándole uno. Se inserta un nuevo registro en la tabla Chat con el identificador generado, el identificador del producto y el nombre de usuario del comprador. El campo archivado se inicializa a cero. Se realiza un commit para confirmar la creación del chat.

- **Enviar Mensaje**

Cuando un usuario envía un mensaje en un chat, el sistema primero verifica que el usuario sea participante del chat. Los participantes válidos son el comprador registrado en la tabla Chat y el vendedor del producto asociado al chat. Si el usuario es participante válido, se inserta un nuevo registro en la tabla Mensaje con el identificador del chat, la fecha y hora actual, el nombre de usuario del emisor, el texto del mensaje, el archivo adjunto si se proporciona, y el campo leído establecido a cero. Se realiza un commit para confirmar el envío del mensaje.

- **Marcar Mensaje como Leído**

Cuando un usuario lee los mensajes de un chat, el sistema actualiza el campo leído a uno para todos los mensajes del chat donde el emisor sea diferente al usuario actual. Esto se realiza mediante una actualización en la tabla Mensaje filtrando por el identificador del chat y el nombre de usuario distinto al del lector. Se realiza un commit para confirmar los cambios.

- **Archivar Chat**

Cuando una venta se confirma su recepción, todos los chats sobre el producto vendido se archivan automáticamente actualizando el campo correspondiente. Esto también se hace si la cuenta de uno de los usuarios que participan se elimina o si el producto deja de estar disponible, ya sea por que se elimina o se vende. Al venderse el único que no se archiva es el chat del comprador. Al hacerse con un trigger en la BD, el commit es el mismo que confirma la recepción.

- **Consultar Lista de Chats**

Cuando un usuario accede a su lista de chats, el sistema consulta la tabla Chat para obtener todos los chats donde el usuario sea el comprador registrado o sea el vendedor del producto asociado. Para cada chat, se consulta la tabla Mensaje para obtener el último mensaje enviado. También se consulta la tabla Producto para obtener información del título del producto asociado. Los chats se ordenan por la fecha del último mensaje, mostrando primero los más recientes.

- **Consultar Histórico de Chats**

Cuando el usuario desea consultar sus chats archivados, se accede a una ventana similar a la de consultar lista de chats, con la única diferencia de que no permite mandar nuevos mensajes.

- **Buscar Mensajes**

Cuando el usuario desee buscar un mensaje por: Emisor, Contenido o Fecha de envío (día concreto); podrá abrir una ventana en la que incluir estos filtros y además decidir si quiere buscar entre los chats archivados también. Sólo se buscará entre los chats a los que pertenezca el usuario. Estos mensajes se mostrarán en una ventana aparte, junto con su fecha y emisor.

7. Disparadores

```
--=====TRIGGERS=====
=====

--FAVORITOS (DANIEL HIDALGO CHICA)

/**
 * TRIGGERS PARA MANTENER SINCRONIZADO EL CONTADOR DE FAVORITOS
 * Responsable: Daniel Hidalgo
 *
 * Sistema de triggers que mantiene actualizado el campo num_favs en la tabla
 Producto.
 *
 * Triggers implementados:
 * 1. TR_Favorito_InsertDisponible: Valida que el producto esté disponible
 * 2. TR_Favorito_InsertUsuarioActivo: Valida que el usuario no está
 eliminado
 * 3. TR_Favorito_Insert: Incrementa num_favs cuando se añade un favorito
 * 4. TR_Favorito_Delete: Decrementa num_favs cuando se elimina un favorito
 * 5. TR_Usuario_SoftDelete: Borra favoritos de usuarios eliminados y ajusta
 contadores
 */

--
=====
-- RECALCULAR CONTADORES INICIALES (ejecución única)
--
=====
-- Esta actualización recalcula el número de favoritos para cada producto.
-- Solo cuenta favoritos de usuarios con cuenta activa (cuenta_eliminada =
0)
--
-- NOTA: Esta sentencia se ejecuta una sola vez al instalar los triggers.
-- Los triggers posteriores mantendrán los contadores actualizados
automáticamente.

UPDATE Producto
SET num_favs = (
    SELECT COUNT(*)
    FROM Favorito f
    JOIN Usuario u ON f.username = u.username
```

```

WHERE f.id_producto = Producto.id_producto
      AND u.cuenta_eliminada = 0
);

--
=====
-- TRIGGER 1: TR_Favorito_InsertDisponible
--
=====
-- Valida que el producto esté disponible antes de permitir la adición a
-- favoritos.
-- Previene race conditions: si otro proceso vende el producto entre la
-- validación
-- en Python y el INSERT, este trigger lo detiene.
--
-- Dispara: BEFORE INSERT ON Favorito
-- Acción: Lanza error si disponible = 0

CREATE OR REPLACE TRIGGER TR_Favorito_InsertDisponible
  BEFORE INSERT ON Favorito
  FOR EACH ROW
DECLARE
  disponible_flag Producto.disponible%TYPE;
BEGIN
  -- Obtener estado disponible del producto
  SELECT disponible INTO disponible_flag FROM Producto
  WHERE id_producto = :NEW.id_producto;

  -- Validar que está disponible
  IF disponible_flag = 0 THEN
    RAISE_APPLICATION_ERROR(-20010,
      'No se puede añadir a favoritos un producto no disponible');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20011, 'El producto no existe');
END;
/

--
=====
-- TRIGGER 2: TR_Favorito_InsertUsuarioActivo
--
=====
-- Valida que el usuario que intenta marcar como favorito no tiene cuenta
-- eliminada.
-- Previene que usuarios eliminados modifiquen sus favoritos.
--

```

```

-- Dispara: BEFORE INSERT ON Favorito
-- Acción: Lanza error si cuenta_eliminada = 1

CREATE OR REPLACE TRIGGER TR_Favorito_InsertUsuarioActivo
  BEFORE INSERT ON Favorito
  FOR EACH ROW
DECLARE
  cuenta_eliminada_flag Usuario.cuenta_eliminada%TYPE;
BEGIN
  -- Obtener estado de eliminación del usuario
  SELECT cuenta_eliminada INTO cuenta_eliminada_flag FROM Usuario
  WHERE username = :NEW.username;

  -- Validar que la cuenta está activa
  IF cuenta_eliminada_flag = 1 THEN
    RAISE_APPLICATION_ERROR(-20013,
      'No puedes realizar esta acción con una cuenta eliminada');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20014, 'El usuario no existe');
END;
/

--
=====
-- TRIGGER 3: TR_Favorito_Insert
--
=====
-- Incrementa num_favs cuando alguien marca un producto como favorito.
--
-- Dispara: AFTER INSERT ON Favorito
-- Acción: UPDATE Producto SET num_favs = num_favs + 1
--
-- Nota: Los triggers BEFORE (1 y 2) garantizan que el usuario y producto
-- son válidos antes de ejecutarse este trigger.

CREATE OR REPLACE TRIGGER TR_Favorito_Insert
  AFTER INSERT ON Favorito
  FOR EACH ROW
BEGIN
  UPDATE Producto
  SET num_favs = num_favs + 1
  WHERE id_producto = :NEW.id_producto;
END;
/

--

```



```

=====
-- TRIGGER 4: TR_Favorito_Delete
--
=====
-- Decrementa num_favs cuando alguien quita un producto de favoritos.
--
-- Dispara: AFTER DELETE ON Favorito
-- Acción: UPDATE Producto SET num_favs = GREATEST(num_favs - 1, 0)
--
-- GREATEST se usa para evitar valores negativos (defensa contra
inconsistencias).

CREATE OR REPLACE TRIGGER TR_Favorito_Delete
AFTER DELETE ON Favorito
FOR EACH ROW
BEGIN
    UPDATE Producto
    SET num_favs = GREATEST(num_favs - 1, 0)
    WHERE id_producto = :OLD.id_producto;
END;
/

--
=====
-- TRIGGER 5: TR_Usuario_SoftDelete
--
=====
-- Cuando se marca un usuario como eliminado (soft-delete, cuenta_eliminada
= 1),
-- se eliminan automáticamente todos sus favoritos.
--
-- Dispara: AFTER UPDATE ON Usuario
-- Condición: OLD.cuenta_eliminada = 0 AND NEW.cuenta_eliminada = 1
-- Acción: DELETE FROM Favorito WHERE username = :NEW.username
--
-- Nota: EL DELETE disparará automáticamente TR_Favorito_Delete para cada
-- fila eliminada, ajustando correctamente los contadores de los productos.

CREATE OR REPLACE TRIGGER TR_Usuario_SoftDelete
AFTER UPDATE ON Usuario
FOR EACH ROW
WHEN (OLD.cuenta_eliminada = 0 AND NEW.cuenta_eliminada = 1)
BEGIN
    DELETE FROM Favorito WHERE username = :NEW.username;
END;
/

--

```

```

=====
-- FIN DE TRIGGERS FAVORITOS
--
=====

-- MENSAJES Y CHATS (AITOR DE LA IGLESIA GARCÍA)

-- =====
-- TRIGGERS PARA GESTIÓN DE CHATS Y ARCHIVADO AUTOMÁTICO
-- (Aitor de La Iglesia García)
-- =====
-- Implementación de restricciones semánticas RS5.1 - RS5.4
-- =====

-- =====
-- RS3.1: AL confirmar la recepción de un producto,
-- se deben archivar todos los chats asociados a ese producto
-- =====
CREATE OR REPLACE TRIGGER TR_Archivado_Al_Confirmar_Recepcion
AFTER UPDATE OF recepcion_confirmada
ON Vendido
FOR EACH ROW
WHEN (NEW.recepcion_confirmada = 1 AND OLD.recepcion_confirmada <> 1)
BEGIN
    UPDATE Chat
    SET archivado = 1
    WHERE id_producto = :NEW.id_producto;
END;
/

-- =====
-- RS3.2: AL marcar un producto como no disponible,
-- se deben archivar los chats de los usuarios que
-- NO lo hayan comprado
-- =====
CREATE OR REPLACE TRIGGER
TR_Archivado_Al_No_Estar_Disponible_Si_No_Lo_Compraste
AFTER UPDATE OF disponible
ON Producto
FOR EACH ROW
WHEN (NEW.disponible = 0 AND OLD.disponible <> 0)
BEGIN
    UPDATE Chat c
    SET c.archivado = 1
    WHERE c.id_producto = :NEW.id_producto
    AND c.username NOT IN (

```

```

        SELECT v.username
        FROM Vendido v
        WHERE v.id_producto = :NEW.id_producto
    );
END;
/

-- =====
-- RS3.3: Al eliminar (marcar como eliminada) una cuenta,
-- se deben archivar todos los chats donde participe
-- ese usuario, como comprador o como vendedor
-- =====
CREATE OR REPLACE TRIGGER TR_Archivado_Al_Eliminar_Usuario
AFTER UPDATE OF cuenta_eliminada
ON Usuario
FOR EACH ROW
WHEN (NEW.cuenta_eliminada = 1 AND OLD.cuenta_eliminada <> 1)
BEGIN
    UPDATE Chat c
    SET c.archivado = 1
    WHERE c.id_chat IN (
        SELECT c.id_chat
        FROM Chat c
        JOIN Producto p ON p.id_producto = c.id_producto
        WHERE p.username = :NEW.username
            OR c.username = :NEW.username
    );
END;
/

-- =====
-- RS3.4: No se puede crear un chat reflexivo:
-- un usuario no puede abrir un chat sobre su propio producto
-- =====
CREATE OR REPLACE TRIGGER TR_No_Chat_Reflexivo
BEFORE INSERT
ON Chat
FOR EACH ROW
DECLARE
    vendedor Producto.username%TYPE;
BEGIN
    -- Obtener el dueño del producto (solo si está disponible)
    SELECT username
    INTO vendedor
    FROM Producto
    WHERE id_producto = :NEW.id_producto
        AND disponible = 1;

```

```

-- Comparar comprador con vendedor
IF vendedor = :NEW.username THEN
    RAISE_APPLICATION_ERROR(-20051, 'No puedes hablar contigo mismo');
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20052, 'El producto no existe o no está
disponible');
END;
/

COMMIT;

-- =====
-- RESUMEN DE TRIGGERS CREADOS
-- =====
--
-- | Trigger                                | RS   | Descripción
-- |-----|-----|-----|
-- | TR_Archivado_AL_Confirmar_Recepcion    | RS3.1| Archiva chats al
confirmar recepción |
-- | TR_Archivado_AL_No_Estar_Disponible_Si_No_Lo_Compraste | RS3.2| Archiva
chats de no compradores al retirar producto |
-- | TR_Archivado_AL_Eliminar_Usuario      | RS3.3| Archiva chats al
eliminar una cuenta |
-- | TR_No_Chat_Reflexivo                  | RS3.4| Impide crear
chats sobre productos propios o no disponibles |
--
-- =====

-- TRIGGERS PARA GESTIÓN DE USUARIOS (ELSA RODRÍGUEZ MACMICHAEL)

-- RS1.11: Un usuario no puede eliminar su cuenta si tiene ventas activas en
curso.
-- Trigger para validar antes de eliminar lógicamente un usuario
-- Se activa cuando se intenta marcar cuenta_eliminada = 1
CREATE OR REPLACE TRIGGER trg_check_user_soft_deletion
BEFORE UPDATE OF cuenta_eliminada ON Usuario
FOR EACH ROW
DECLARE
    active_sales INT;
    active_offers INT;
BEGIN

```

```

-- Solo validar cuando se está eliminando la cuenta (pasando de 0 a 1)
IF :OLD.cuenta_eliminada = 0 AND :NEW.cuenta_eliminada = 1 THEN
    -- Check for active sales (como vendedor o comprador)
    SELECT COUNT(*) INTO active_sales
    FROM Vendido
    WHERE (Vendido.username = :OLD.username AND recepcion_confirmada = 0)
        OR (Vendido.id_producto IN (
            SELECT id_producto
            FROM Producto
            WHERE username = :OLD.username
        ) AND recepcion_confirmada = 0);

    -- Check for active counteroffers
    SELECT COUNT(*) INTO active_offers
    FROM Contraoferta
    WHERE username = :OLD.username;

    -- Raise an error if there are active sales or counteroffers
    IF active_sales > 0 OR active_offers > 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'No se puede eliminar el usuario
porque tiene ventas activas o contraofertas pendientes.');
```

END IF;

END IF;

END;

/

-- RS1.9: La cantidad a transferir debe estar comprendida entre 0 y el saldo disponible.

-- Trigger para validar que el saldo no sea negativo al actualizar

```

CREATE OR REPLACE TRIGGER trg_check_saldo_positivo
BEFORE UPDATE OF saldo ON Usuario
FOR EACH ROW
BEGIN
    IF :NEW.saldo < 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'El saldo del monedero no puede ser
negativo.');
```

END IF;

END;

/

-- RS1.2, RS1.3: El correo electrónico debe tener formato válido y ser único.

-- Trigger para validar el formato del correo electrónico

```

CREATE OR REPLACE TRIGGER trg_check_email_format
BEFORE INSERT OR UPDATE OF correo ON Usuario
FOR EACH ROW
BEGIN
    IF :NEW.correo IS NOT NULL AND NOT REGEXP_LIKE(:NEW.correo,
```

```

'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$') THEN
    RAISE_APPLICATION_ERROR(-20003, 'El formato del correo electrónico no
es válido.');
```

END IF;

END;

/

-- RS1.17, RS1.18: El rango de interés debe ser positivo.

-- Trigger para validar que el rango de búsqueda sea positivo

CREATE OR REPLACE TRIGGER trg_check_rango_positivo

BEFORE INSERT OR UPDATE OF rango ON Usuario

FOR EACH ROW

BEGIN

IF :NEW.rango IS NOT NULL AND :NEW.rango < 0 THEN

RAISE_APPLICATION_ERROR(-20004, 'El rango de búsqueda debe ser un

valor positivo.');

END IF;

END;

/

-- Trigger para validar las coordenadas de ubicación

CREATE OR REPLACE TRIGGER trg_check_ubicacion_valida

BEFORE INSERT OR UPDATE ON Usuario

FOR EACH ROW

BEGIN

-- Validar Latitud (entre -90 y 90)

IF :NEW.ubi_latitud IS NOT NULL AND (:NEW.ubi_latitud < -90 OR

:NEW.ubi_latitud > 90) THEN

RAISE_APPLICATION_ERROR(-20005, 'La latitud debe estar entre -90 y 90

grados.');

END IF;

-- Validar Longitud (entre -180 y 180)

IF :NEW.ubi_longitud IS NOT NULL AND (:NEW.ubi_longitud < -180 OR

:NEW.ubi_longitud > 180) THEN

RAISE_APPLICATION_ERROR(-20006, 'La longitud debe estar entre -180 y

180 grados.');

END IF;

END;

/

-- Trigger para validar la valoración media (entre 0 y 5)

CREATE OR REPLACE TRIGGER trg_check_valoracion_rango

BEFORE INSERT OR UPDATE OF valoracion_media ON Usuario

FOR EACH ROW

BEGIN

IF :NEW.valoracion_media IS NOT NULL AND (:NEW.valoracion_media < 0 OR

:NEW.valoracion_media > 5) THEN

```

        RAISE_APPLICATION_ERROR(-20007, 'La valoración media debe estar entre
0 y 5.');
```

END IF;

END;

/

-- RS1.6, RS1.7: Un usuario no puede tener menos de 1 ni más de 6 categorías de preferencia.

-- Compound trigger para evitar error de tabla mutante (ORA-04091)

CREATE OR REPLACE TRIGGER trg_check_num_categorias

FOR INSERT ON Preferidos

COMPOUND TRIGGER

-- Colección para almacenar los usuarios afectados

TYPE t_usernames IS TABLE OF Preferidos.username%TYPE INDEX BY PLS_INTEGER;

v_usernames t_usernames;

v_index PLS_INTEGER := 0;

AFTER EACH ROW IS

BEGIN

-- Guardar el username para verificar después

v_index := v_index + 1;

v_usernames(v_index) := :NEW.username;

END AFTER EACH ROW;

AFTER STATEMENT IS

num_cats INT;

BEGIN

-- Verificar cada usuario afectado

FOR i IN 1..v_usernames.COUNT LOOP

SELECT COUNT(*) INTO num_cats

FROM Preferidos

WHERE username = v_usernames(i);

IF num_cats > 6 THEN

RAISE_APPLICATION_ERROR(-20008, 'Un usuario no puede tener más de 6 categorías preferidas.');

END IF;

END LOOP;

END AFTER STATEMENT;

END trg_check_num_categorias;

/

-- RS1.12 a RS1.16: Ninguna operación de gestión de perfil podrá ejecutarse si el usuario ha sido eliminado.

-- Trigger para evitar modificar usuarios eliminados

CREATE OR REPLACE TRIGGER trg_check_cuenta_eliminada

BEFORE UPDATE ON Usuario

```

FOR EACH ROW
BEGIN
    -- Si la cuenta está eliminada, solo permitir cambios en cuenta_eliminada
    (para reactivar)
    IF :OLD.cuenta_eliminada = 1 AND :NEW.cuenta_eliminada = 1 THEN
        RAISE_APPLICATION_ERROR(-20009, 'No se puede modificar una cuenta que
ha sido eliminada.');
```

```

    END IF;
END;
/

--TRIGGERS PARA GESTIÓN DE PRODUCTOS (ROBERTO GONZALEZ LUGO)
-- =====
-- TRIGGERS PARA GESTIÓN DE PRODUCTOS (Roberto González)
-- =====
-- Implementación de restricciones semánticas RS2.1 - RS2.15
-- =====

-- =====
-- RS2.1 / RS2.2: El precio debe ser mayor que 0
-- Aplica a INSERT y UPDATE en Producto
-- =====
CREATE OR REPLACE TRIGGER trg_check_precio_producto
BEFORE INSERT OR UPDATE OF precio ON Producto
FOR EACH ROW
BEGIN
    IF :NEW.precio IS NULL OR :NEW.precio <= 0 THEN
        RAISE_APPLICATION_ERROR(-20101, 'El precio del producto debe ser
mayor que 0.');
```

```

    END IF;
END;
/

-- =====
-- RS2.3 / RS2.4: La categoría debe existir en el sistema
-- Esto ya está garantizado por la FK, pero añadimos
-- mensaje de error más descriptivo
-- =====
CREATE OR REPLACE TRIGGER trg_check_categoria_producto
BEFORE INSERT OR UPDATE OF nombre_categoria ON Producto
FOR EACH ROW
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM Categoria
    WHERE nombre = :NEW.nombre_categoria;
```



```

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20102, 'La categoría seleccionada no existe
en el sistema.');
```

END IF;

END;

/

-- =====

-- RS2.5 / RS2.6: Longitudes máximas de título (80) y descripción (500)

-- =====

```

CREATE OR REPLACE TRIGGER trg_check_longitud_producto
BEFORE INSERT OR UPDATE ON Producto
FOR EACH ROW
BEGIN
    -- Verificar Longitud del título (máximo 80 caracteres)
    IF :NEW.titulo IS NOT NULL AND LENGTH(:NEW.titulo) > 80 THEN
        RAISE_APPLICATION_ERROR(-20103, 'El título no puede exceder los 80
caracteres.');
```

END IF;

-- Verificar Longitud de la descripción (máximo 500 caracteres)

```

    IF :NEW.descripcion IS NOT NULL AND LENGTH(:NEW.descripcion) > 500 THEN
        RAISE_APPLICATION_ERROR(-20104, 'La descripción no puede exceder los
500 caracteres.');
```

END IF;

END;

/

-- =====

-- RS2.8: Solo se puede modificar un producto disponible

-- y sin contraofertas activas

-- =====

```

CREATE OR REPLACE TRIGGER trg_check_modificar_producto
BEFORE UPDATE ON Producto
FOR EACH ROW
DECLARE
    v_contraofertas INT;
```

BEGIN

-- Solo aplicar si se están modificando campos de contenido (no promoción

ni num_favs)

```

    IF :OLD.titulo != :NEW.titulo
    OR :OLD.descripcion != :NEW.descripcion
    OR :OLD.precio != :NEW.precio
    OR :OLD.nombre_categoria != :NEW.nombre_categoria
    OR (:OLD.imagen IS NULL AND :NEW.imagen IS NOT NULL)
    OR (:OLD.imagen IS NOT NULL AND :NEW.imagen IS NULL) THEN
```

-- Verificar que el producto esté disponible

```

        IF :OLD.disponible = 0 THEN
            RAISE_APPLICATION_ERROR(-20105, 'No se puede modificar un
producto que no está disponible.');
```

END IF;

-- Verificar que no tenga contraofertas activas

```

SELECT COUNT(*) INTO v_contraofertas
FROM Contraoferta
WHERE id_producto = :OLD.id_producto;

        IF v_contraofertas > 0 THEN
            RAISE_APPLICATION_ERROR(-20106, 'No se puede modificar un
producto con contraofertas activas.');
```

END IF;

END IF;

END;

/

-- =====

-- RS2.9: Solo se puede eliminar (marcar no disponible)

-- un producto disponible sin contraofertas activas

-- =====

```

CREATE OR REPLACE TRIGGER trg_check_eliminar_producto
BEFORE UPDATE OF disponible ON Producto
FOR EACH ROW
DECLARE
    v_contraofertas INT;
BEGIN
    -- Solo aplicar cuando se está marcando como no disponible
    IF :OLD.disponible = 1 AND :NEW.disponible = 0 THEN
        -- Verificar que no tenga contraofertas activas
        SELECT COUNT(*) INTO v_contraofertas
        FROM Contraoferta
        WHERE id_producto = :OLD.id_producto;

        IF v_contraofertas > 0 THEN
            RAISE_APPLICATION_ERROR(-20107, 'No se puede eliminar un producto
con contraofertas activas.');
```

END IF;

END IF;

END;

/

-- =====

-- RS2.10: El grado de promoción debe estar en [0, 1]

-- =====

```

CREATE OR REPLACE TRIGGER trg_check_promocion_rango
BEFORE INSERT OR UPDATE OF promocion ON Producto
```

```

FOR EACH ROW
BEGIN
    IF :NEW.promocion IS NOT NULL THEN
        IF :NEW.promocion < 0 THEN
            :NEW.promocion := 0;
        ELSIF :NEW.promocion > 1 THEN
            RAISE_APPLICATION_ERROR(-20108, 'El grado de promoción debe estar
entre 0 y 1.');
```

-- =====

-- RS2.11: Decaimiento automático de promoción

-- (0.1 cada 24 horas - se implementaría con un JOB)

-- Este trigger evitaría valores negativos

-- =====

```

CREATE OR REPLACE TRIGGER trg_promocion_no_negativa
BEFORE UPDATE OF promocion ON Producto
FOR EACH ROW
BEGIN
    -- Asegurar que la promoción nunca sea negativa
    IF :NEW.promocion IS NOT NULL AND :NEW.promocion < 0 THEN
        :NEW.promocion := 0;
    END IF;
END;
/

-- =====
-- RS2.12 / RS2.13: id_producto se genera automáticamente
-- y es único (implementado con secuencia)
-- =====

CREATE SEQUENCE seq_producto_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER trg_auto_id_producto
BEFORE INSERT ON Producto
FOR EACH ROW
BEGIN
    -- Asignar ID automático si no se proporciona o es nulo
    IF :NEW.id_producto IS NULL THEN
        SELECT seq_producto_id.NEXTVAL INTO :NEW.id_producto FROM DUAL;
    END IF;
END;
/

```

```

/

-- =====
-- RS2.14: El vendedor debe existir y no estar eliminado
-- =====

CREATE OR REPLACE TRIGGER trg_check_vendedor_existe
BEFORE INSERT ON Producto
FOR EACH ROW
DECLARE
    v_eliminado INT;
BEGIN
    SELECT cuenta_eliminada INTO v_eliminado
    FROM Usuario
    WHERE username = :NEW.username;

    IF v_eliminado = 1 THEN
        RAISE_APPLICATION_ERROR(-20109, 'No se puede crear un producto para
un usuario con cuenta eliminada.');
```

END IF;

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20110, 'El vendedor especificado no
existe.');
```

END;

```

/

-- =====
-- Trigger adicional: Inicializar valores por defecto
-- =====

CREATE OR REPLACE TRIGGER trg_producto_defaults
BEFORE INSERT ON Producto
FOR EACH ROW
BEGIN
    -- Promoción por defecto a 0 si es nula
    IF :NEW.promocion IS NULL THEN
        :NEW.promocion := 0;
    END IF;

    -- num_favs por defecto a 0 si es nulo
    IF :NEW.num_favs IS NULL THEN
        :NEW.num_favs := 0;
    END IF;

    -- disponible por defecto a 1 (disponible) si es nulo
    IF :NEW.disponible IS NULL THEN
        :NEW.disponible := 1;
    END IF;
END;
```

```

/

COMMIT;

-- =====
-- RESUMEN DE TRIGGERS CREADOS
-- =====
--
-- | Trigger                                | RS      | Descripción
-- |-----|-----|-----|
-- | trg_check_precio_producto             | RS2.1/2 | Precio > 0
-- | trg_check_categoria_producto          | RS2.3/4 | Categoría debe existir
-- | trg_check_Longitud_producto           | RS2.5/6 | Título ≤80, Descripción ≤500
-- | trg_check_modificar_producto          | RS2.8   | No modificar si no disponible
-- | trg_check_eliminar_producto           | RS2.9   | No eliminar si tiene
-- | trg_check_promocion_rango              | RS2.10  | Promoción en [0, 1]
-- | trg_promocion_no_negativa              | RS2.11  | Evita promoción negativa
-- | trg_auto_id_producto                   | RS2.12/13 | ID automático y único
-- | trg_check_vendedor_existe              | RS2.14  | Vendedor existe y no eliminado
-- | trg_producto_defaults                  | -       | Valores por defecto
--
-- =====

--TRIGGERS PARA GESTIÓN DE VENTAS (JUAN MANUEL FERNÁNDEZ GARCÍA)
/**
* TRIGGERS PARA MANTENER CONSISTENTES LAS TABLAS VENDIDO Y CONTRAOFERTA.
* Responsable: Juanma Fernández
*
* Sistema de triggers que evitan que se inserten tuplas inconsistentes en
las tablas
* Vendido y Contraoferta.
*
* Triggers implementados:
* 1. TR_Vendido_Insert: Evita que se inserten ventas en las que el comprador
es el dueño del producto.

```

```

* 2. TR_Contraoferta_Insertar: Evita que se inserten contraofertas en las
que el comprador es el dueño
* del producto.
* 3. TR_Update_Contraofertas: Elimina las contraofertas no válidas asociadas
a un producto cuando
* se actualiza un producto en venta.
*/

--
=====
-- TRIGGER 1: TR_Vendido_Insert
--
=====
-- Evita que se inserte en la tabla VENDIDO una venta en la que el usuario
comprador
-- es dueño de su propio producto.
--
-- Dispara: BEFORE INSERT ON VENDIDO
-- Acción: Comprobar que el usuario comprador no sea el dueño del producto
que ha comprado
--
-- Nota: En el código que implementa la compra directa y la aceptación de
contraofertas ya se
-- comprueba antes de realizar una consulta que el comprador no sea dueño
del producto y que el
-- producto existe, por lo que este disparador no debería lanzar nunca
ninguna excepción.

CREATE OR REPLACE TRIGGER TR_Vendido_Insert
  BEFORE
  INSERT ON Vendido
  FOR EACH ROW
DECLARE
  propietario Producto.username%TYPE;
BEGIN
  -- Obtener dueño del producto
  SELECT username INTO propietario FROM PRODUCTO
  WHERE id_producto = :new.id_producto;

  -- Comparar comprador con propietario
  IF propietario = :new.username THEN
    RAISE_APPLICATION_ERROR(-20001, 'No puedes comprar tu propio
producto');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20002, 'El producto no existe');
END;

```

```

/

--
=====
-- TRIGGER 2: TR_Contraoferta_Insertar
--
=====
-- Evita que se inserte en la tabla CONTRAOFERTA una contraoferta en la que
el
-- usuario comprador es dueño de su propio producto.
--
-- Dispara: BEFORE INSERT ON CONTRAOFERTA
-- Acción: Comprobar que el usuario comprador no sea el dueño del producto
al que realiza la contraoferta
--
-- Nota: En el código que implementa la aceptación de contraofertas ya se
comprueba antes de realizar
-- una consulta que el comprador no sea dueño del producto y que el producto
existe, por lo que este disparador
-- no debería lanzar nunca ninguna excepción.

CREATE OR REPLACE TRIGGER TR_Contraoferta_Insertar
  BEFORE
  INSERT ON Contraoferta
  FOR EACH ROW
DECLARE
  propietario Producto.username%TYPE;
BEGIN
  -- Obtener dueño del producto
  SELECT username INTO propietario FROM PRODUCTO
  WHERE id_producto = :new.id_producto;

  -- Comparar comprador con propietario
  IF propietario = :new.username THEN
    RAISE_APPLICATION_ERROR(-20003, 'No puedes realizar contraofertas a
tu propio producto');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20002, 'El producto no existe');
END;
/

-- =====

```

8. Software elegido

Para el desarrollo de TikiTun hemos optado por utilizar Python como lenguaje de programación principal, combinado con Tkinter para la interfaz gráfica y Oracle Database como sistema gestor de bases de datos, accedido a través de SQL Developer para la administración y el desarrollo de los scripts SQL.

La elección de Python se fundamenta en su versatilidad y facilidad de uso, lo que permite un desarrollo ágil de prototipos funcionales. Además, al ser un lenguaje ampliamente utilizado en el ámbito académico, todos los miembros del equipo contábamos con experiencia previa en él, lo que ha facilitado la colaboración y reducido la curva de aprendizaje.

Tkinter ha sido seleccionado como biblioteca para la interfaz gráfica por venir incluido en la distribución estándar de Python, eliminando la necesidad de instalar dependencias externas complejas. Aunque existen alternativas más modernas y visualmente atractivas, Tkinter ofrece la funcionalidad suficiente para construir una interfaz de usuario completa y operativa, permitiéndonos centrarnos en la lógica de negocio y la interacción con la base de datos sin distraernos con aspectos puramente estéticos.

En cuanto al sistema gestor de bases de datos, hemos utilizado Oracle Database siguiendo las indicaciones del Seminario 1 de la asignatura, donde se introdujo su uso junto con SQL Developer como herramienta de administración. Esta continuidad nos ha permitido aprovechar los conocimientos adquiridos durante las sesiones prácticas y aplicarlos directamente al desarrollo del proyecto. SQL Developer nos ha facilitado la creación y depuración de las tablas, triggers y procedimientos almacenados, así como la inserción de datos de prueba y la verificación de las consultas antes de integrarlas en el código Python.

La conexión entre Python y Oracle se realiza mediante la biblioteca oracledb, que proporciona una interfaz compatible con el estándar DB-API de Python, permitiendo ejecutar sentencias SQL y gestionar transacciones de forma sencilla y consistente con lo aprendido en la asignatura.

9. Datos sensibles y aspectos legales

- **Identificación de datos sensibles:**

La base de datos de TikiTun almacena diversas categorías de información personal que requieren especial protección conforme a la normativa vigente en materia de protección de datos.

En primer lugar, el sistema recopila datos identificativos básicos de los usuarios, incluyendo el nombre de usuario, la dirección de correo electrónico y la contraseña de acceso. Aunque estos datos pueden parecer rutinarios, constituyen información personal que permite

identificar de manera única a cada individuo dentro de la plataforma y, en el caso del correo electrónico, también fuera de ella.

En segundo lugar, la aplicación almacena datos de localización geográfica de los usuarios. Esta información resulta especialmente sensible porque permite conocer la ubicación física de las personas, lo cual podría utilizarse de manera indebida si cayera en manos equivocadas. La localización se emplea en TikiTun para facilitar las transacciones entre usuarios cercanos y calcular distancias entre compradores y vendedores.

En tercer lugar, el sistema gestiona datos de carácter económico relacionados con el monedero virtual de cada usuario. Esto incluye el saldo disponible y el historial de transacciones realizadas dentro de la plataforma. Aunque no se almacenan datos bancarios externos como números de tarjeta o cuentas corrientes, la información económica interna sigue siendo considerada sensible por revelar patrones de consumo y capacidad económica de los usuarios.

Adicionalmente, la plataforma conserva las comunicaciones privadas entre usuarios a través del sistema de chat integrado. Estos mensajes pueden contener información personal, acuerdos comerciales, datos de contacto alternativos u otra información que los usuarios compartan en el contexto de sus negociaciones, por lo que su protección resulta fundamental.

Por último, se almacenan datos relativos a las preferencias y comportamiento de los usuarios, tales como sus categorías favoritas, productos marcados como favoritos, valoraciones emitidas y recibidas, y el historial de productos publicados y adquiridos. Esta información permite construir un perfil detallado de cada usuario que debe ser protegido adecuadamente.

- **Medidas implementadas en el interfaz**

Para cumplir con las obligaciones legales en materia de protección de datos, la aplicación incorpora diversos mecanismos informativos y de obtención de consentimiento en su interfaz de usuario.

Durante el proceso de registro, antes de que el usuario pueda crear su cuenta, se le presenta una casilla de verificación mediante la cual debe aceptar expresamente la política de privacidad de la plataforma. Esta casilla no aparece marcada por defecto, obligando al usuario a realizar una acción afirmativa que constituye prueba de su consentimiento informado, requisito indispensable según la normativa para que la inclusión de datos personales en la base de datos sea legítima. Junto a dicha casilla se incluye un enlace que permite acceder al texto completo de la política de privacidad, donde se detallan todos los aspectos relativos al tratamiento de sus datos personales.

La política de privacidad accesible desde el registro informa al usuario sobre la identidad del responsable del tratamiento, los tipos de datos que se recopilan, las finalidades para las que se utilizan, la base legal que legitima el tratamiento, los destinatarios de los datos, los plazos de conservación y, de manera destacada, los derechos que asisten al usuario en relación con sus datos personales.

En cuanto a los derechos del usuario, la política de privacidad informa detalladamente sobre el derecho de acceso, que permite al usuario conocer qué datos suyos están siendo tratados; el derecho de rectificación, que le faculta para corregir datos inexactos o incompletos; el derecho de supresión, que le permite solicitar la eliminación de sus datos cuando ya no sean necesarios; el derecho de oposición, que le habilita para oponerse a determinados tratamientos; el derecho a la portabilidad, que le permite obtener sus datos en un formato estructurado; y el derecho a la limitación del tratamiento, que le faculta para restringir el uso de sus datos en determinadas circunstancias.

- **Trámites legales y organismos competentes**

El tratamiento de datos personales en España está regulado principalmente por el Reglamento General de Protección de Datos de la Unión Europea y por la Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales. El cumplimiento de esta normativa requiere la realización de diversos trámites legales ante los organismos competentes.

A nivel nacional, el organismo de control es la Agencia Española de Protección de Datos, entidad pública independiente encargada de velar por el cumplimiento de la legislación sobre protección de datos y de garantizar el derecho fundamental a la protección de datos personales. Ante este organismo deben realizarse los principales trámites relacionados con el registro y la gestión de las actividades de tratamiento, y es también ante esta agencia donde los usuarios pueden presentar reclamaciones si consideran que sus derechos han sido vulnerados.

A nivel europeo, el Comité Europeo de Protección de Datos coordina la aplicación coherente del Reglamento General de Protección de Datos en todos los Estados miembros de la Unión Europea, emitiendo directrices y dictámenes que deben ser tenidos en cuenta por las autoridades nacionales y por los responsables del tratamiento.

En el ámbito universitario, la Universidad de Granada dispone de su propia Oficina de Protección de Datos, que vela por el cumplimiento de la normativa en el tratamiento de datos personales dentro de la institución. Aunque TikiTun es un proyecto académico, el conocimiento de estas estructuras resulta relevante para comprender el marco institucional de la protección de datos.

Toda base de datos que contenga información de personas debe contar con un responsable del tratamiento claramente identificado. Este responsable es la persona física o jurídica que determina los fines y medios del tratamiento y sobre quien recaen las obligaciones legales de protección de datos. En el caso de TikiTun, el responsable del tratamiento queda identificado en la política de privacidad con sus datos de contacto correspondientes.

El responsable del tratamiento debe elaborar y mantener actualizado un Registro de Actividades de Tratamiento, documento interno que describe de manera detallada todos los tratamientos de datos personales que se llevan a cabo, incluyendo las finalidades, las categorías de datos e interesados, los destinatarios, las transferencias internacionales si las hubiera, los plazos de conservación y las medidas de seguridad aplicadas. Aunque este

registro no se presenta ante ningún organismo, debe estar disponible para la Agencia Española de Protección de Datos en caso de inspección.

Cuando el tratamiento pueda entrañar un alto riesgo para los derechos y libertades de las personas físicas, resulta obligatorio realizar una Evaluación de Impacto en la Protección de Datos antes de iniciar dicho tratamiento. Esta evaluación permite identificar y mitigar los riesgos potenciales, y en determinados casos debe ser consultada con la Agencia Española de Protección de Datos antes de proceder al tratamiento.

- **Seguridad de los datos**

Un aspecto fundamental en la gestión de bases de datos con información personal es la seguridad de los datos. El responsable del tratamiento debe implementar medidas técnicas y organizativas apropiadas para garantizar un nivel de seguridad adecuado al riesgo, incluyendo el control de acceso a los datos y la prevención de su robo, alteración o destrucción no autorizada.

En TikiTun, las contraseñas de los usuarios se almacenan actualmente en texto plano en la base de datos. Somos conscientes de que esta práctica no es recomendable en un entorno de producción real, ya que en caso de una brecha de seguridad las contraseñas quedarían expuestas directamente. Lo adecuado sería implementar funciones de hash criptográfico como SHA-256 o bcrypt, que transforman la contraseña en una representación irreversible antes de almacenarla. De este modo, incluso si un atacante accediera a la base de datos, no podría recuperar las contraseñas originales. Esta simplificación se ha adoptado en el contexto de este proyecto académico para evitar complicaciones adicionales en el desarrollo y las pruebas, pero en un sistema real destinado a usuarios reales sería imprescindible implementar el hash de contraseñas.

El acceso a la base de datos está restringido mediante credenciales específicas y la comunicación con el servidor de base de datos se realiza de forma controlada, limitando las operaciones que pueden ejecutarse desde la interfaz de usuario.

Finalmente, en caso de producirse una violación de seguridad que afecte a datos personales, el responsable del tratamiento tiene la obligación de notificarla a la Agencia Española de Protección de Datos en un plazo máximo de setenta y dos horas desde que tenga conocimiento de ella, salvo que sea improbable que dicha violación constituya un riesgo para los derechos y libertades de las personas físicas. Si la violación entraña un alto riesgo, también debe comunicarse a los usuarios afectados sin dilación indebida.